

TERRAFORM TASK - 06

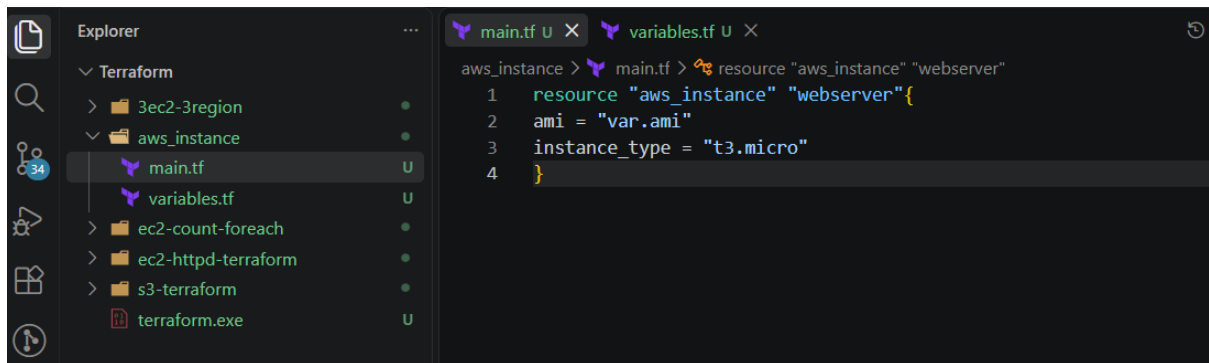
1. Execute the script shown in the video.

What is a Terraform Module?

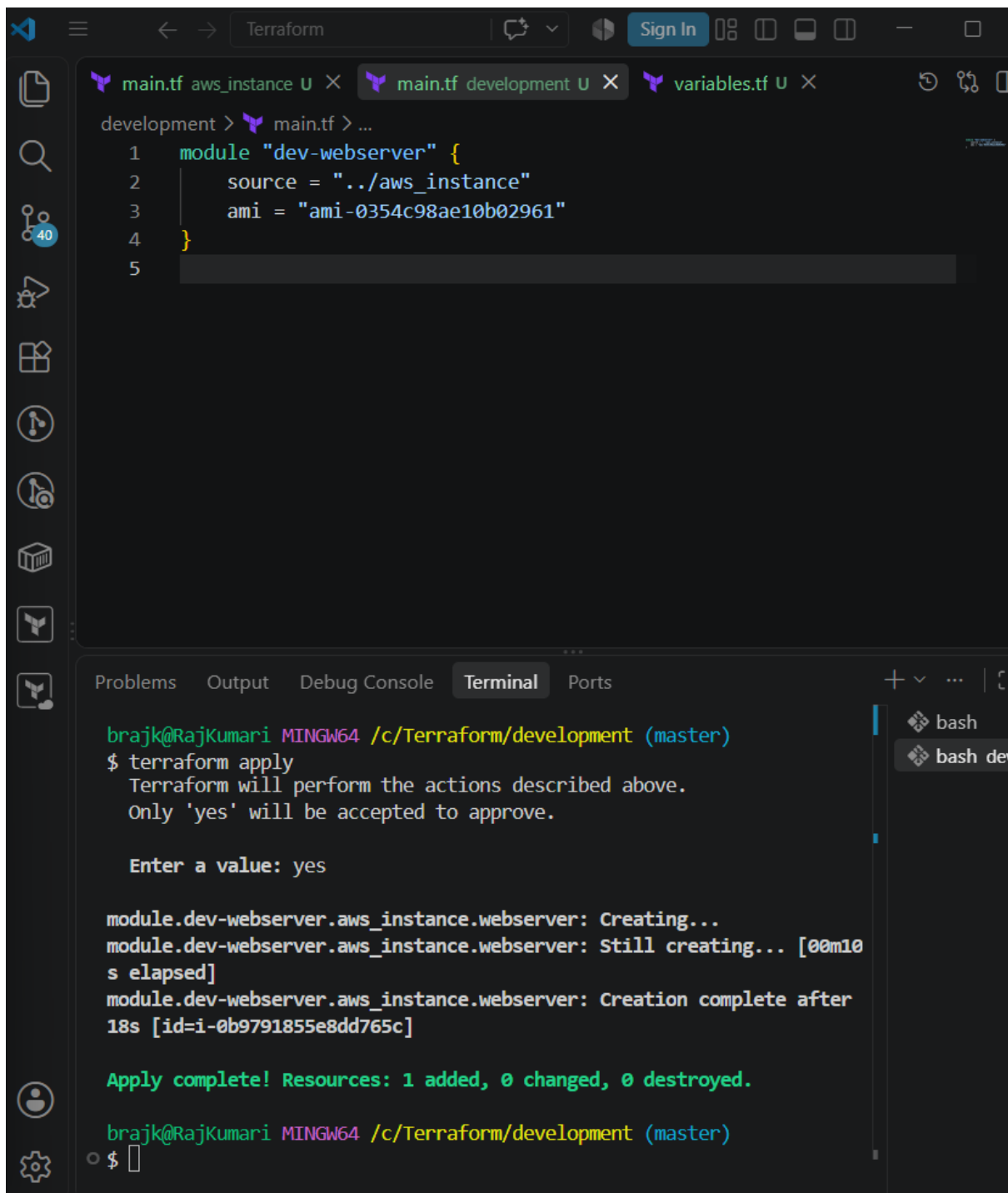
A **Terraform module** is a **reusable collection of Terraform configuration files** that are used to create and manage infrastructure.

In simple words: **A module is like a reusable template of Terraform code.**

Instead of writing the same Terraform code again and again, we create a module once and reuse it multiple times.

A screenshot of a code editor interface. On the left, the 'Explorer' sidebar shows a file tree with a folder named 'Terraform' containing subfolders '3ec2-3region', 'aws_instance', 'ec2-count-foreach', 'ec2-httpd-terraform', 's3-terraform', and a file 'terraform.exe'. The 'aws_instance' folder is expanded, showing 'main.tf' and 'variables.tf'. The 'main.tf' file is selected. On the right, the editor shows the content of 'main.tf', which defines an AWS instance resource named 'webserver'. The code is as follows:

```
aws_instance > main.tf > resource "aws_instance" "webserver"
1 resource "aws_instance" "webserver"{
2   ami = "var.ami"
3   instance_type = "t3.micro"
4 }
```



AWS Management Console - EC2 Instances

Instances (1) Info

Filter Instance by attribute or tag (case-sensitive)

Instance state = running

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
	i-0b9791855e8dd765c	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-54-83-250-14

All instances

Instance alarm states

In alarm: None

Service health

EC2 service status: Operating normally

Health events affecting EC2 instances

Open events: None

Upcoming events: None

VS Code Explorer

main.tf development U

main.tf production U

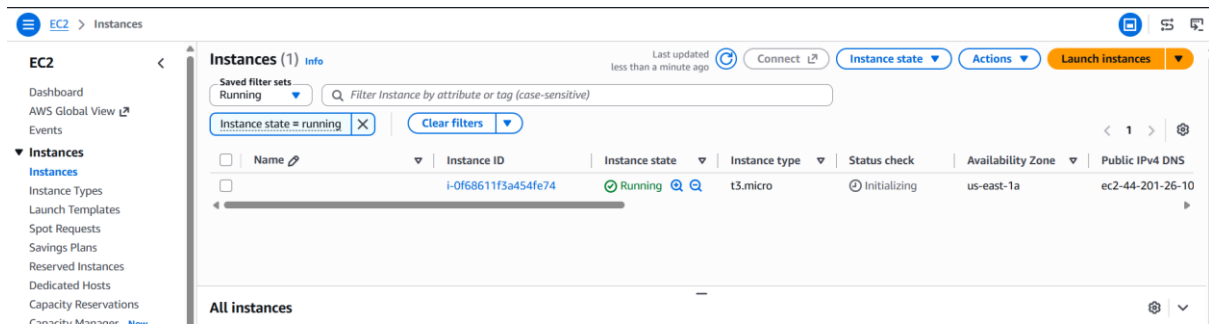
```
production > main.tf > ...
1 module "dev-webserver" {
2     source = "../aws_instance"
3     ami = "ami-0354c98ae10b02961"
4 }
5
```

Terminal

```
brajk@RajKumari MINGW64 /c/Terraform/production (master)
$ terraform apply

module.dev-webserver.aws_instance.webserver: Creating...
module.dev-webserver.aws_instance.webserver: Still creating... [00m10s elapsed]
module.dev-webserver.aws_instance.webserver: Creation complete after 19s [id=i-0f68611f3a454fe74]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed
```



What is a Terraform Workspace?

A **Terraform workspace** allows you to use the **same Terraform configuration** to manage **different Terraform state files**.

In simple words:

A workspace is a separate environment with its own Terraform state.

For example, you can use the same code for:

Development

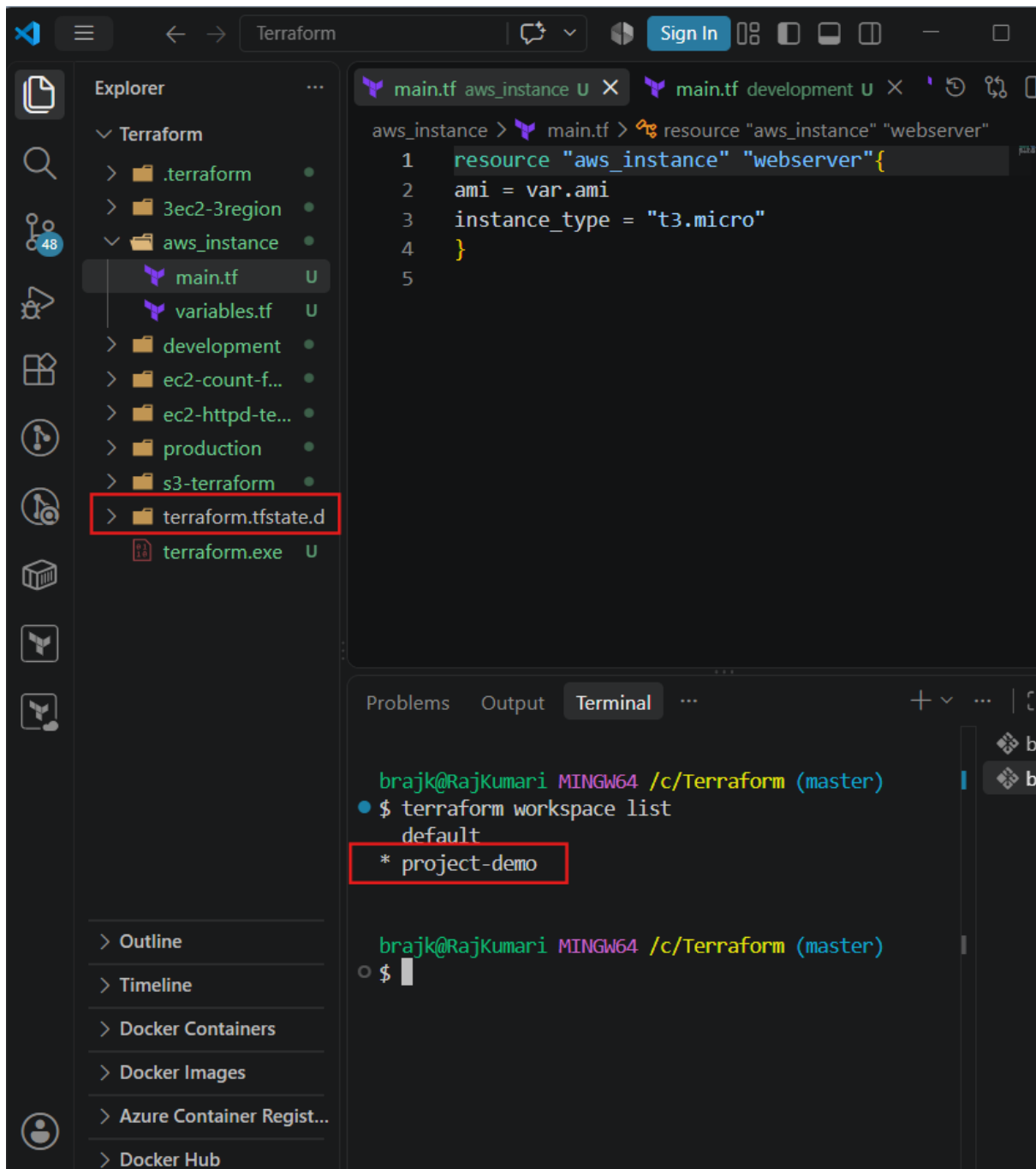
↓

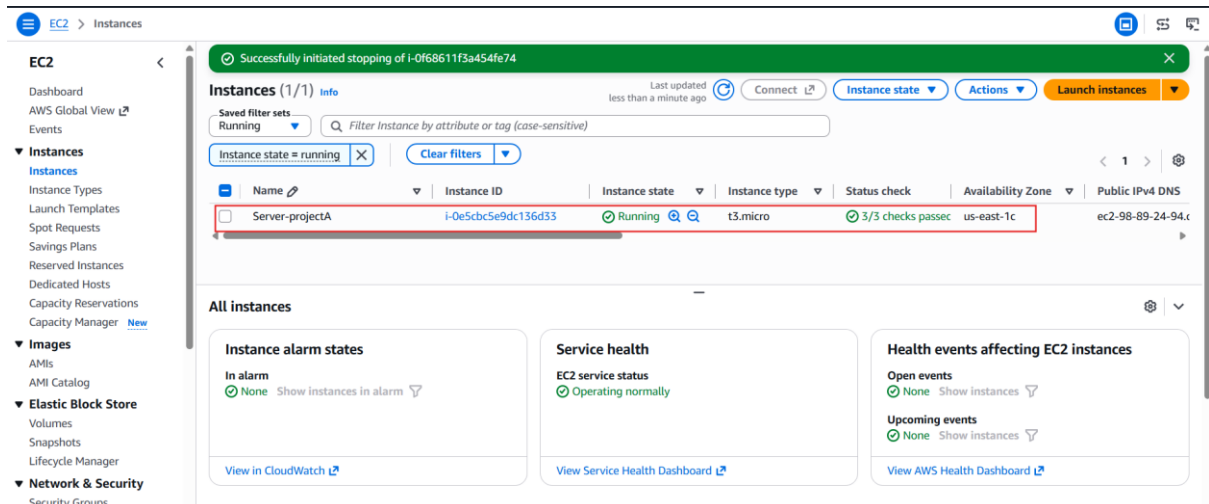
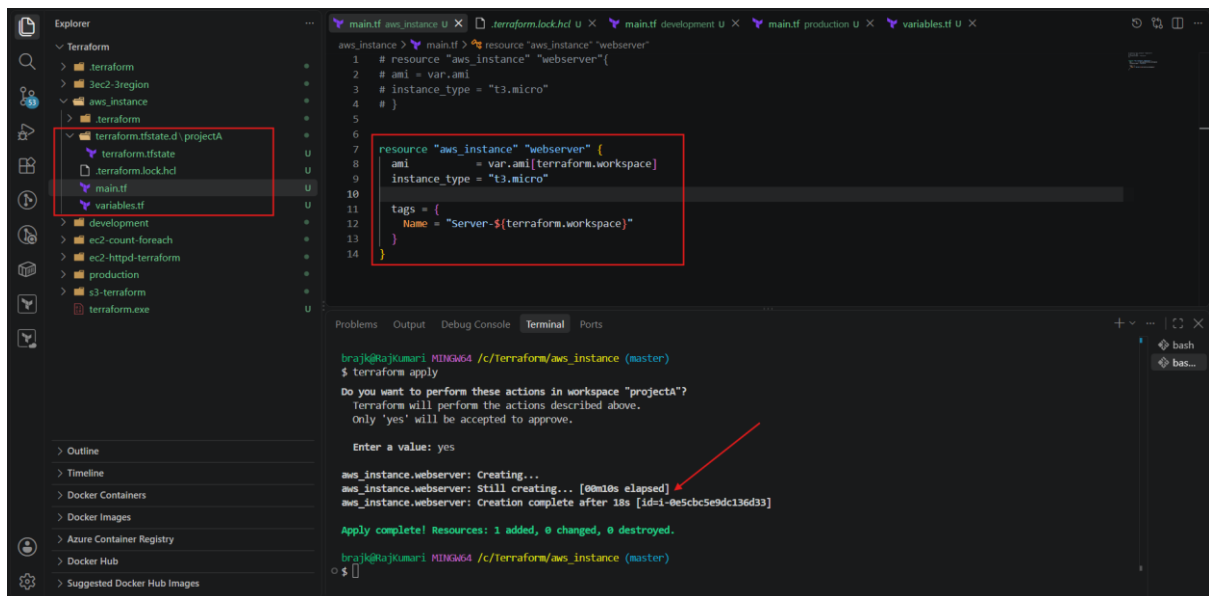
Testing

↓

Production

without creating separate Terraform code for each environment.





2.Provision EC2, S3, and VPC using Terraform modules.

Terraform Modules project where we provision:

1. VPC
2. S3 Bucket
3. EC2 Instance

The important concept is that each infrastructure component will be a separate reusable module.

1. Final architecture

We will create:

Terraform Project

|

├── provider.tf

├── variables.tf

├── main.tf

├── outputs.tf

├── terraform.tfvars

|

└── modules/

|

├── vpc/

| ├── main.tf

| ├── variables.tf

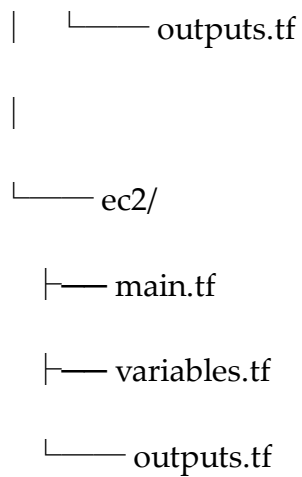
| └── outputs.tf

|

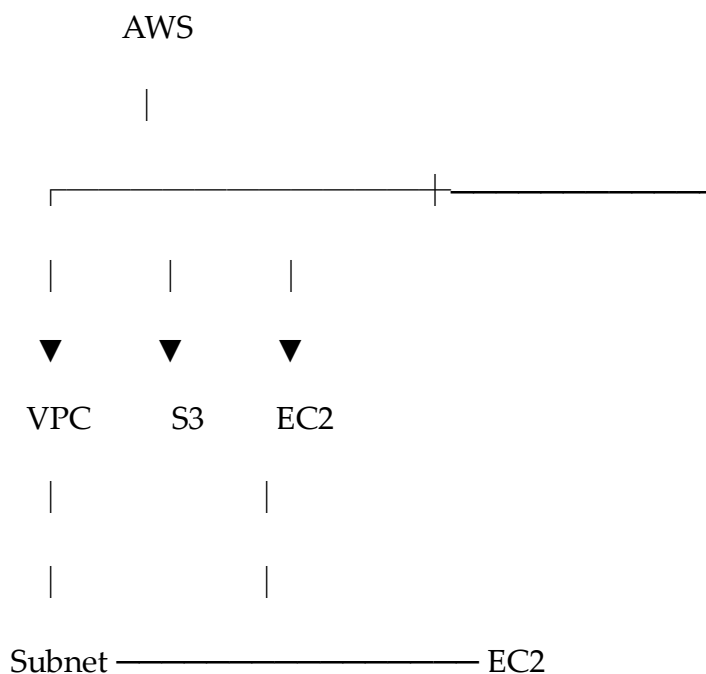
├── s3/

| ├── main.tf

| └── variables.tf



The final AWS architecture will be:



2. Create the project directory

Since you're using Git Bash on Windows, run:

```
cd /c/Terraform
```

Create the project:

```
mkdir terraform-modules-project
```

```
cd terraform-modules-project
```

Create the module directories:

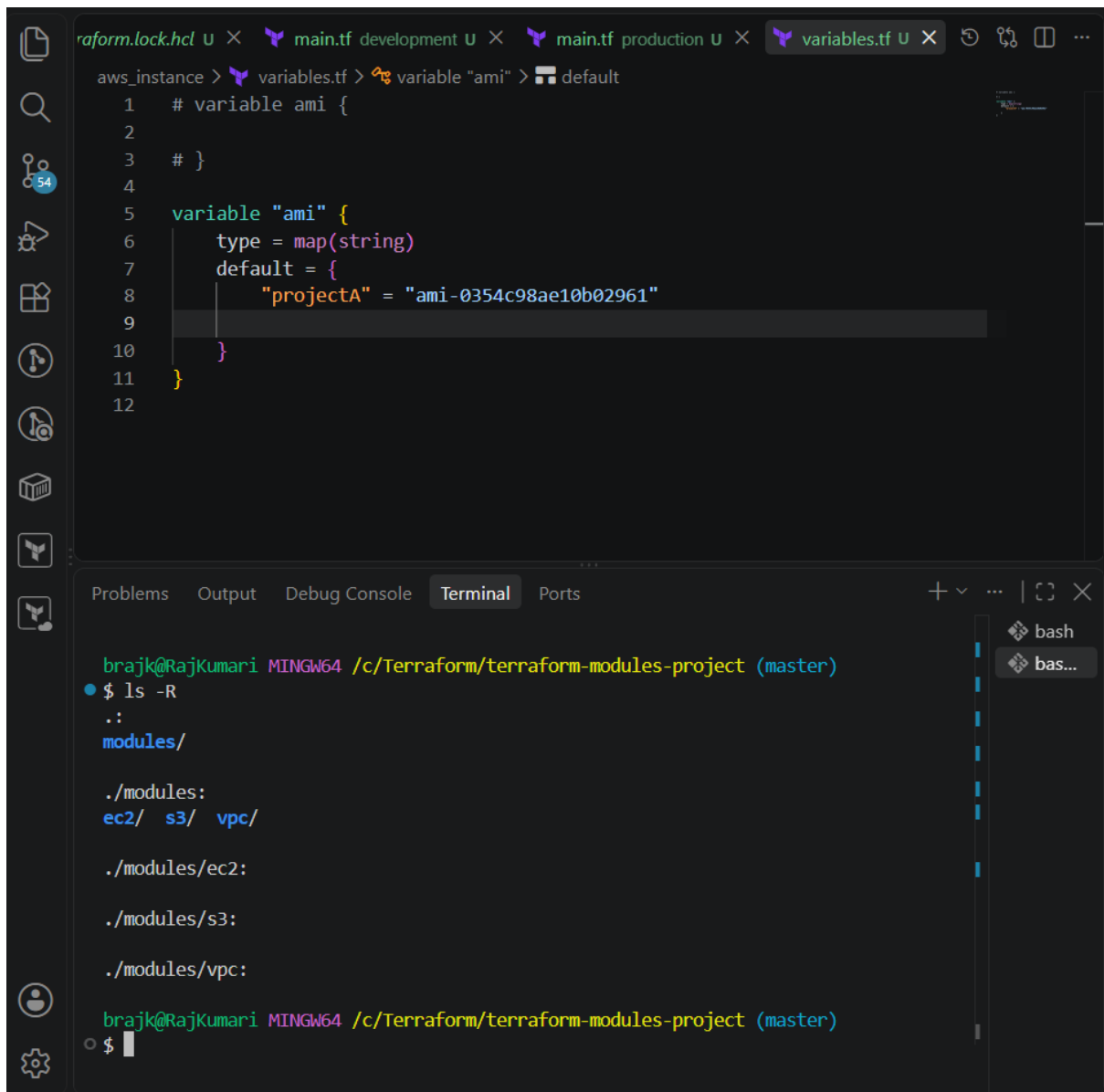
```
mkdir -p modules/vpc
```

```
mkdir -p modules/s3
```

```
mkdir -p modules/ec2
```

Check:

```
ls -R
```



The screenshot shows a VS Code editor with several tabs: `raform.lock.hcl`, `main.tf development`, `main.tf production`, and `variables.tf`. The `variables.tf` file is open, showing a Terraform variable definition for `ami`. The variable is of type `map(string)` and has a default value for `projectA` set to `"ami-0354c98ae10b02961"`. Below the editor, the `Terminal` panel is active, showing the output of the `ls -R` command. The output shows the directory structure: `modules/`, `./modules/`, `ec2/ s3/ vpc/`, `./modules/ec2:`, `./modules/s3:`, and `./modules/vpc:`. The terminal prompt is `brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)`.

```
aws_instance > variables.tf > variable "ami" > default
1  # variable ami {
2
3  # }
4
5  variable "ami" {
6      type = map(string)
7      default = {
8          "projectA" = "ami-0354c98ae10b02961"
9      }
10 }
11
12

brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$ ls -R
.:
modules/

./modules:
ec2/  s3/  vpc/

./modules/ec2:

./modules/s3:

./modules/vpc:
```

4. Configure AWS provider

Create:

```
provider.tf
```

Add:

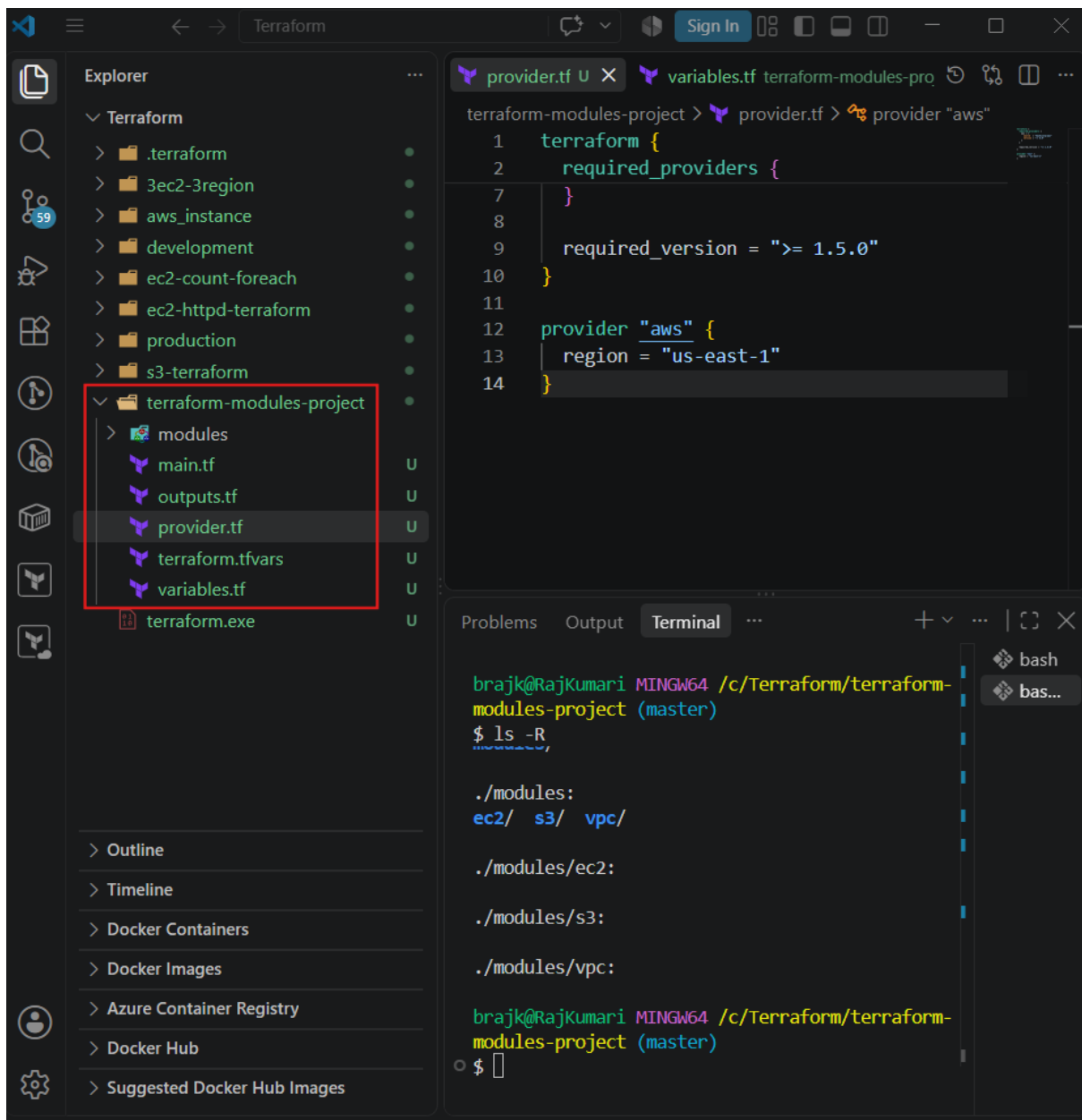
```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
  
  required_version = ">= 1.5.0"  
}
```

```
provider "aws" {  
  region = "us-east-1"  
}
```

We're using:

us-east-1

for this lab.



5. Create the VPC module

Go to:

```
cd modules/vpc
```

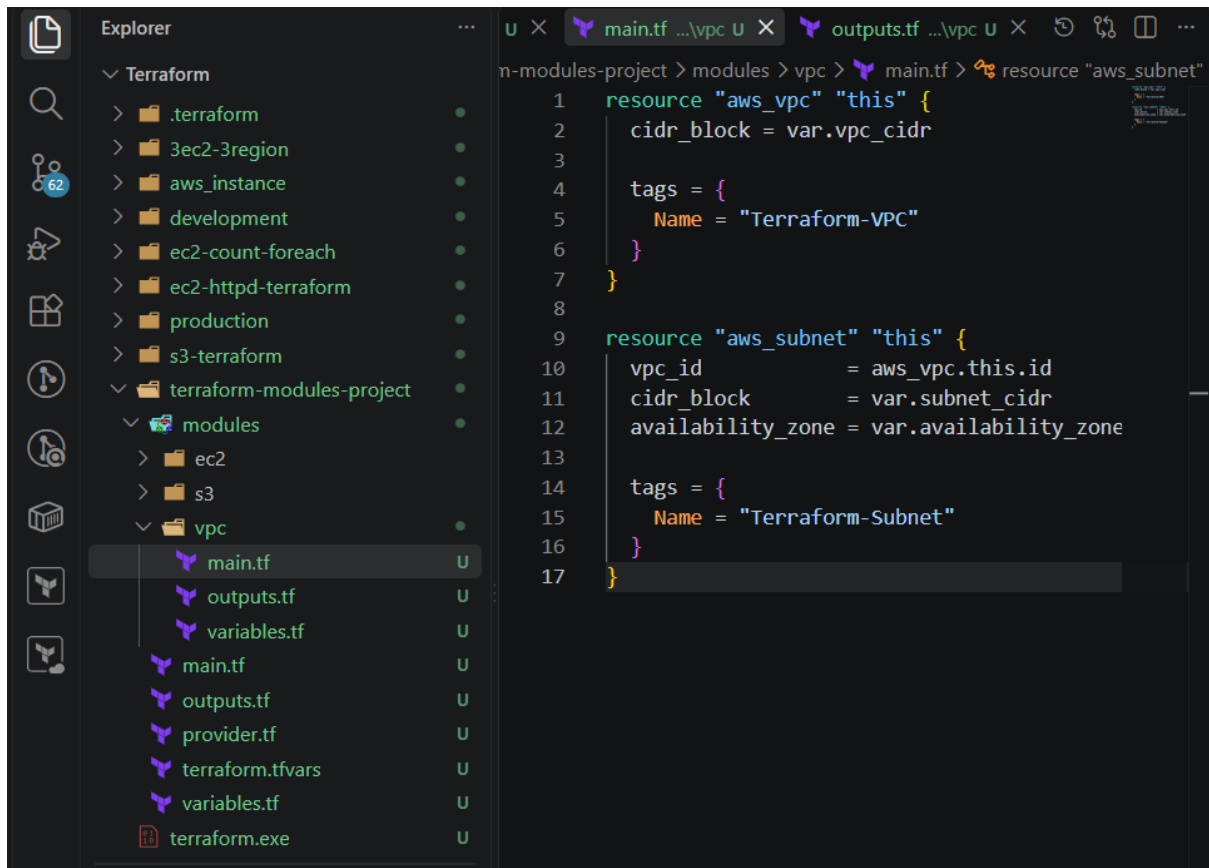
Create:

```
main.tf
```

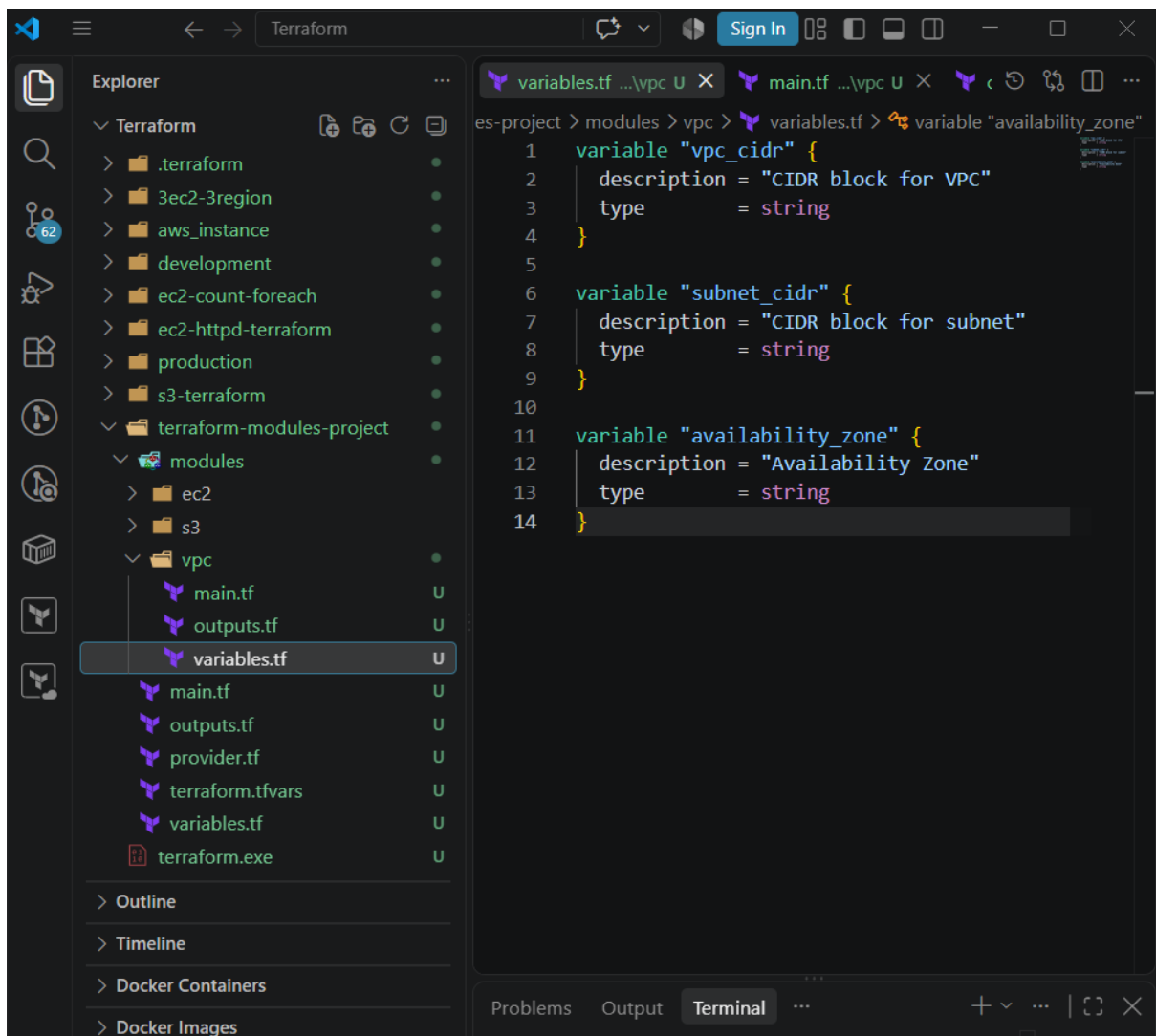
```
variables.tf
```

```
outputs.tf
```

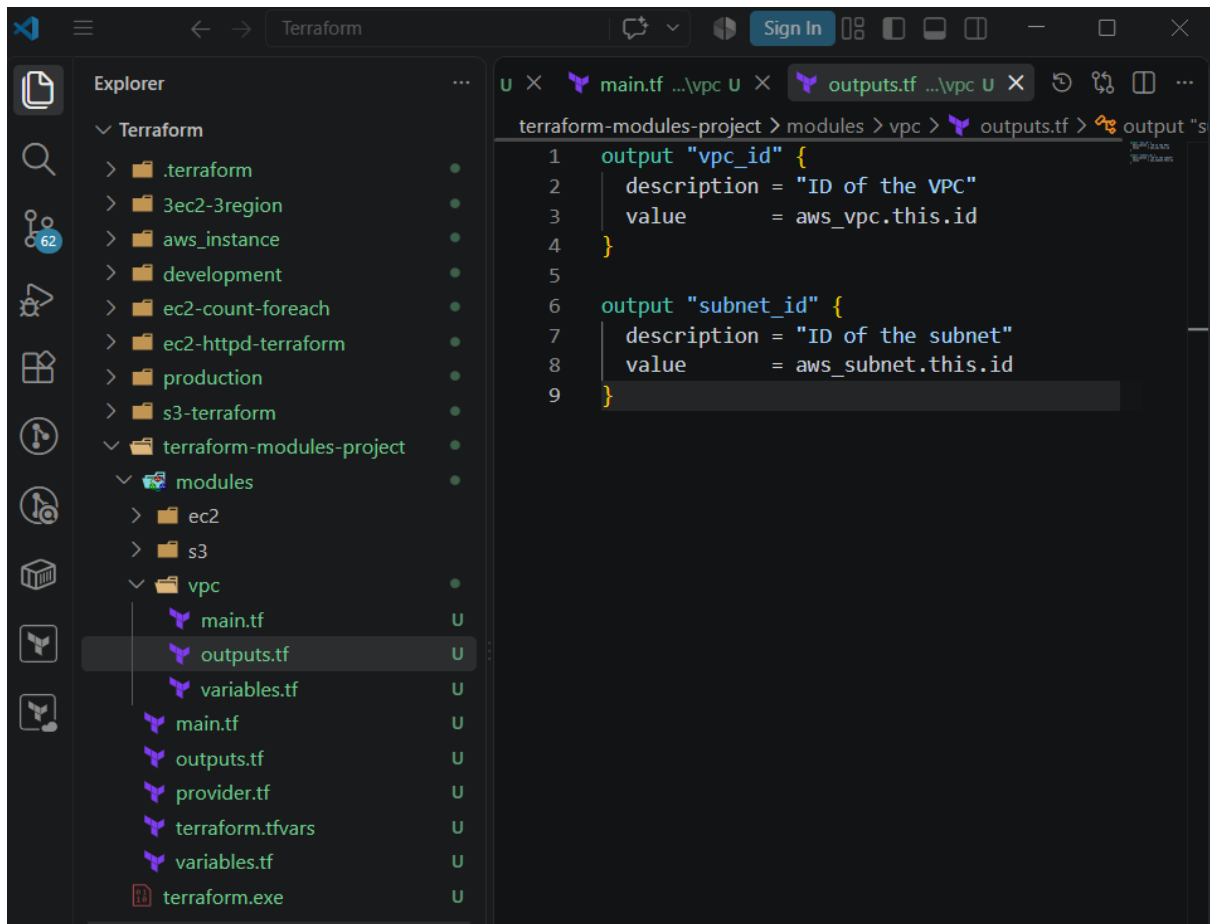
5.1 VPC module main.tf



5.2 VPC module variables.tf



5.3 VPC module outputs.tf



6. Create the S3 module

Go back to the project root:

```
cd ../../
```

Go to S3:

```
cd modules/s3
```

Create:

main.tf

variables.tf

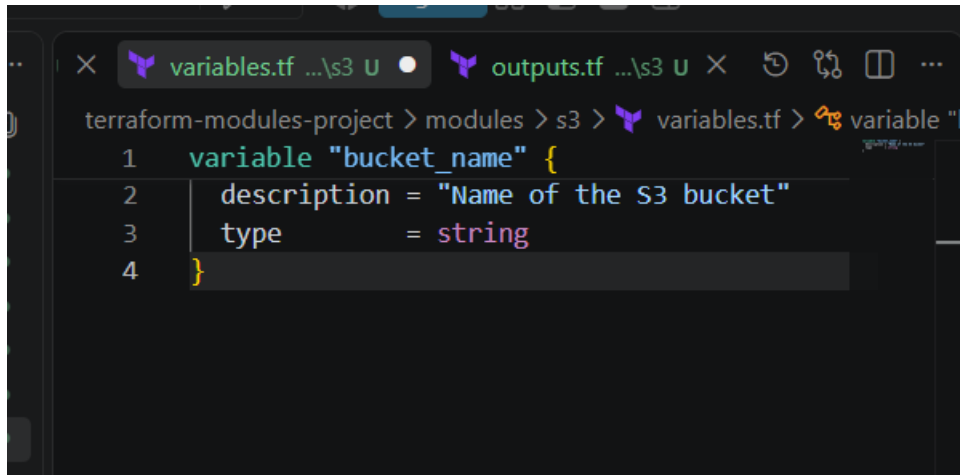
outputs.tf

6.1 S3 module variables.tf

```
variable "bucket_name" {
```

```
description = "Name of the S3 bucket"

type      = string
}
```



6.2 S3 module main.tf

```
resource "aws_s3_bucket" "this" {

  bucket = var.bucket_name


  tags = {

    Name      = var.bucket_name

    Environment = "Dev"
  }
}
```

A screenshot of a code editor with a dark theme. The editor shows a Terraform configuration file named `main.tf` within a project structure `terraform-modules-project > modules > s3 > main.tf`. The code defines an `aws_s3_bucket` resource named `this`. It sets the `bucket` attribute to `var.bucket_name` and defines a `tags` block with `Name` set to `var.bucket_name` and `Environment` set to `"Dev"`. The code is as follows:

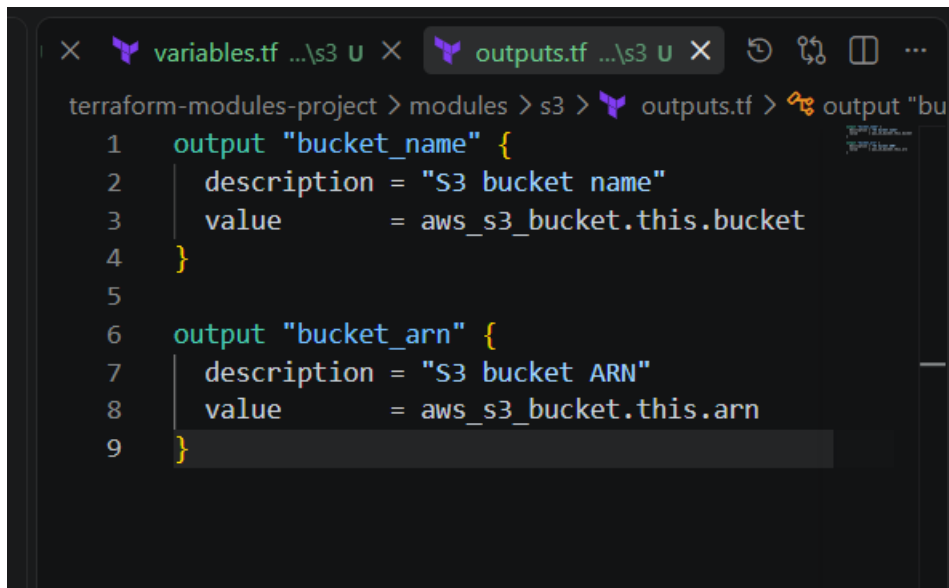
```
1 resource "aws_s3_bucket" "this" {
2     bucket = var.bucket_name
3
4     tags = {
5         Name      = var.bucket_name
6         Environment = "Dev"
7     }
8 }
```

6.3 S3 module outputs.tf

```
output "bucket_name" {
    description = "S3 bucket name"
    value      = aws_s3_bucket.this.bucket
}
```

```
output "bucket_arn" {
    description = "S3 bucket ARN"
    value      = aws_s3_bucket.this.arn
}
```

The S3 module is now complete.



```
terraform-modules-project > modules > s3 > outputs.tf > output "bu
1  output "bucket_name" {
2      description = "S3 bucket name"
3      value       = aws_s3_bucket.this.bucket
4  }
5
6  output "bucket_arn" {
7      description = "S3 bucket ARN"
8      value       = aws_s3_bucket.this.arn
9  }
```

7. Create the EC2 module

Go back:

```
cd ../../
```

Then:

```
cd modules/ec2
```

Create:

```
main.tf
```

```
variables.tf
```

```
outputs.tf
```

7.1 EC2 module variables.tf

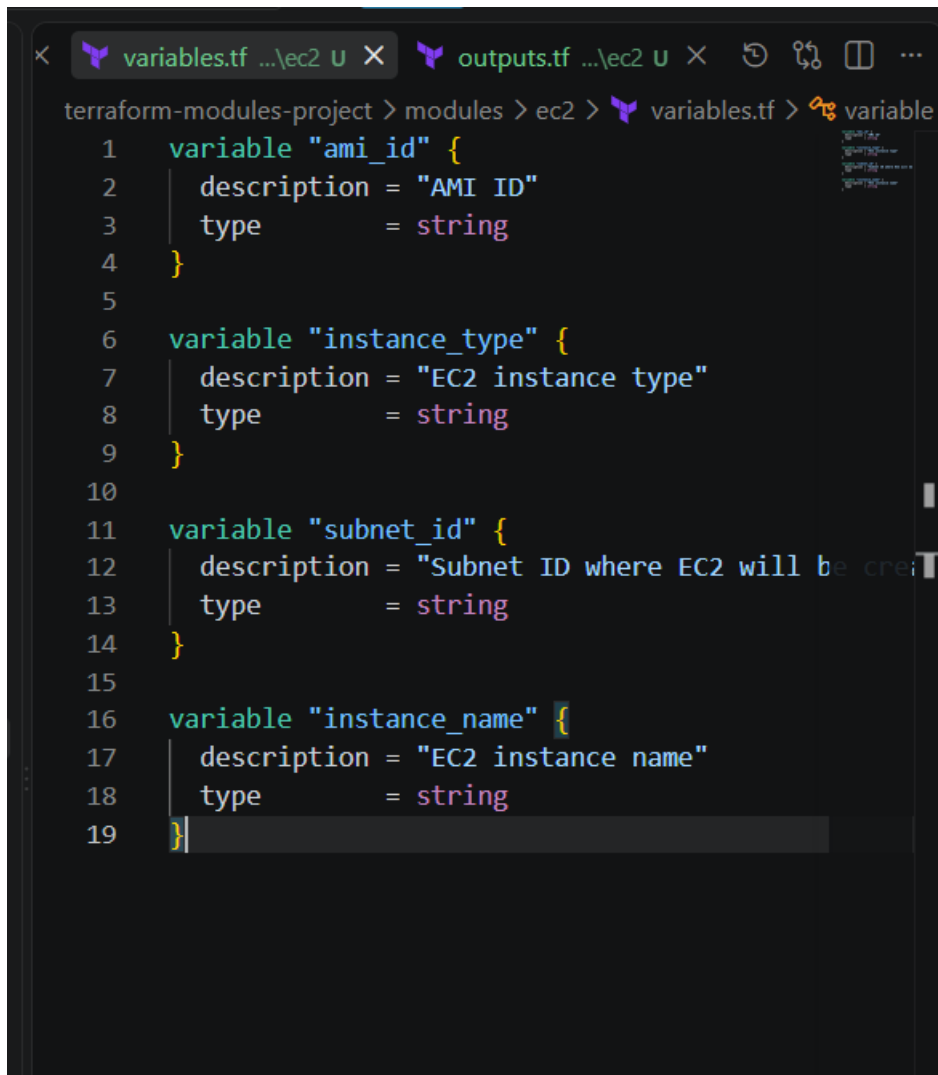
```
variable "ami_id" {
    description = "AMI ID"
    type       = string
}
```

```
variable "instance_type" {
```

```
description = "EC2 instance type"
type      = string
}
```

```
variable "subnet_id" {
  description = "Subnet ID where EC2 will be created"
  type      = string
}
```

```
variable "instance_name" {
  description = "EC2 instance name"
  type      = string
}
```

A screenshot of a code editor with a dark theme. The editor shows a Terraform file named 'variables.tf' within a project structure 'terraform-modules-project > modules > ec2 > variables.tf'. The code defines four variables: 'ami_id', 'instance_type', 'subnet_id', and 'instance_name'. Each variable is a string type with a descriptive comment. The editor has tabs for 'variables.tf' and 'outputs.tf'.

```
1 variable "ami_id" {
2     description = "AMI ID"
3     type        = string
4 }
5
6 variable "instance_type" {
7     description = "EC2 instance type"
8     type        = string
9 }
10
11 variable "subnet_id" {
12     description = "Subnet ID where EC2 will be created"
13     type        = string
14 }
15
16 variable "instance_name" {
17     description = "EC2 instance name"
18     type        = string
19 }
```

8. EC2 module main.tf

```
resource "aws_security_group" "this" {
    name      = "${var.instance_name}-sg"
    description = "Security group for EC2"
    vpc_id    = data.aws_vpc.selected.id

    ingress {
        description = "SSH"
        from_port   = 22
        to_port     = 22
    }
}
```

```
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

```
egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}
```

```
tags = {
    Name = "${var.instance_name}-SG"
}
}
```

```
data "aws_vpc" "selected" {
    id = data.aws_subnet.selected.vpc_id
}
```

```
data "aws_subnet" "selected" {
    id = var.subnet_id
}
```

```
resource "aws_instance" "this" {
    ami          = var.ami_id
```

```
instance_type      = var.instance_type
subnet_id          = var.subnet_id
vpc_security_group_ids = [aws_security_group.this.id]
```

```
tags = {
  Name = var.instance_name
}
}
```

This module uses the subnet provided by the VPC module.

The relationship is:

VPC Module

|

| subnet_id



EC2 Module

|



EC2 Instance

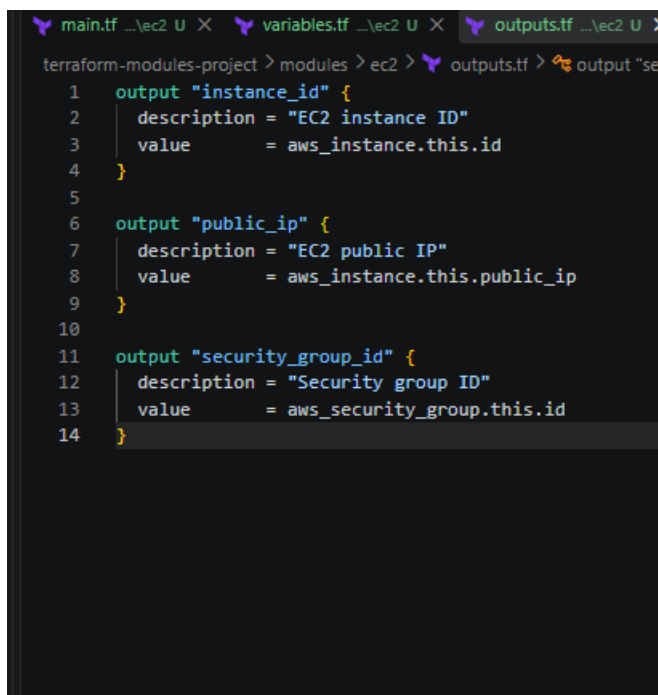
```
main.tf ...\ec2 U  variables.tf ...\ec2 U X  outputs.tf ...\ec2 U X  variables.tf ter
terraform-modules-project > modules > ec2 > main.tf > resource "aws_instance" "this"
1  resource "aws_security_group" "this" {
2      name           = "${var.instance_name}-sg"
3      description    = "Security group for EC2"
4      vpc_id         = data.aws_vpc.selected.id
5
6      ingress {
7          description = "SSH"
8          from_port   = 22
9          to_port     = 22
10         protocol    = "tcp"
11         cidr_blocks = ["0.0.0.0/0"]
12     }
13
14     egress {
15         from_port   = 0
16         to_port     = 0
17         protocol    = "-1"
18         cidr_blocks = ["0.0.0.0/0"]
19     }
20
21     tags = {
22         Name = "${var.instance_name}-SG"
23     }
24 }
25
26 data "aws_vpc" "selected" {
27     id = data.aws_subnet.selected.vpc_id
28 }
29
30 data "aws_subnet" "selected" {
31     id = var.subnet_id
32 }
33
34 resource "aws_instance" "this" {
35     ami                = var.ami_id
36     instance_type      = var.instance_type
37     subnet_id          = var.subnet_id
38     vpc_security_group_ids = [aws_security_group.this.id]
39
40     tags = {
41         Name = var.instance_name
42     }
43 }
```

9. EC2 module outputs.tf

```
output "instance_id" {
    description = "EC2 instance ID"
    value       = aws_instance.this.id
}
```

```
output "public_ip" {  
    description = "EC2 public IP"  
    value      = aws_instance.this.public_ip  
}
```

```
output "security_group_id" {  
    description = "Security group ID"  
    value      = aws_security_group.this.id  
}
```

A screenshot of a code editor with a dark theme. The editor shows a file named 'outputs.tf' with line numbers 1 through 14. The code defines three Terraform outputs: 'instance_id', 'public_ip', and 'security_group_id'. Each output block includes a description and a value. The 'instance_id' output uses 'aws_instance.this.id', 'public_ip' uses 'aws_instance.this.public_ip', and 'security_group_id' uses 'aws_security_group.this.id'. The editor's breadcrumb navigation at the top shows the path: 'terraform-modules-project > modules > ec2 > outputs.tf'.

```
1  output "instance_id" {  
2      description = "EC2 instance ID"  
3      value      = aws_instance.this.id  
4  }  
5  
6  output "public_ip" {  
7      description = "EC2 public IP"  
8      value      = aws_instance.this.public_ip  
9  }  
10  
11 output "security_group_id" {  
12     description = "Security group ID"  
13     value      = aws_security_group.this.id  
14 }
```

10. Go back to root

```
cd ../../
```

You should be here:

```
/c/Terraform/terraform-modules-project
```

11. Root variables.tf

Create:

```
variables.tf
```

```
in.tf ...\ec2 U X variables.tf ...\ec2 U X outputs.tf ...\ec2 U X variables.tf terraform-modules-proj
terraform-modules-project > variables.tf > variable "instance_type" > abc description
1 variable "vpc_cidr" {
2   type      = string
3   description = "VPC CIDR"
4 }
5
6 variable "subnet_cidr" {
7   type      = string
8   description = "Subnet CIDR"
9 }
10
11 variable "availability_zone" {
12   type      = string
13   description = "Availability Zone"
14 }
15
16 variable "bucket_name" {
17   type      = string
18   description = "S3 bucket name"
19 }
20
21 variable "ami_id" {
22   type      = string
23   description = "AMI ID"
24 }
25
26 variable "instance_type" {
27   type      = string
28   description = "EC2 instance type"
29 }
30
31 variable "instance_name" {
32   type      = string
33   description = "EC2 instance name"
34 }
```

12. Root terraform.tfvars

Create:

terraform.tfvars

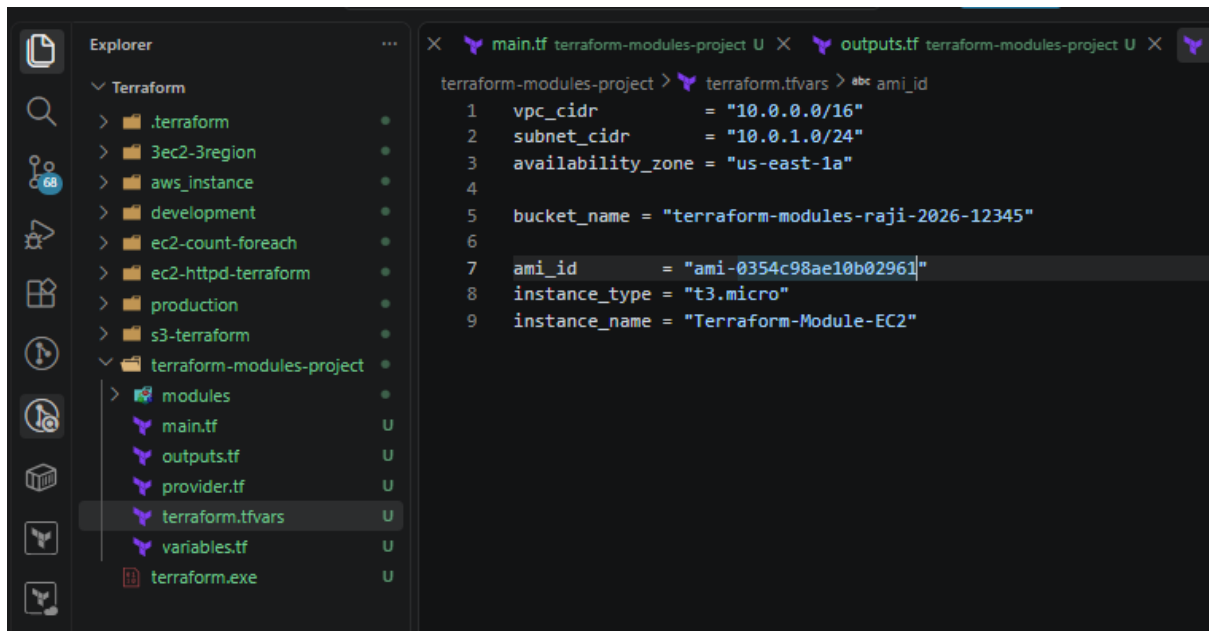
Important

Replace:

YOUR_AMI_ID

with a valid Amazon Linux AMI in us-east-1.

Also, the S3 bucket name must be **globally unique**.



13. Root main.tf — call all modules

This is the most important file.

Notice this line:

```
subnet_id = module.vpc.subnet_id
```

This is extremely important.

It means:

Get the subnet ID created by the VPC module and pass it to the EC2 module.

So Terraform understands:

VPC Module

```
|  
└── subnet_id
```

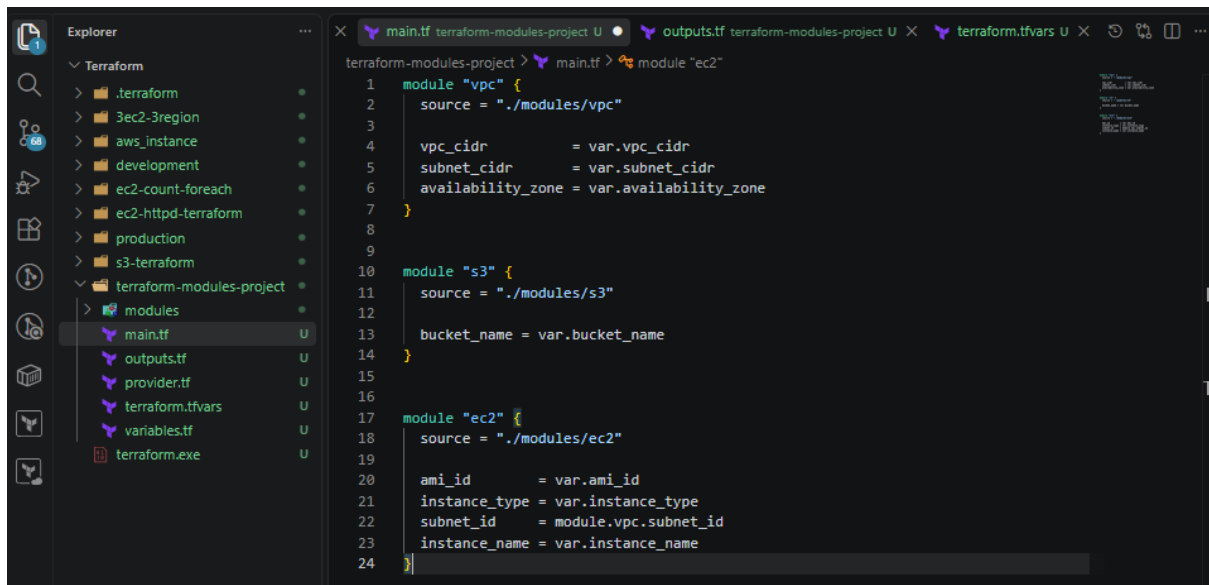
```
|  
▼
```

EC2 Module

```
|  
▼
```

EC2 Instance

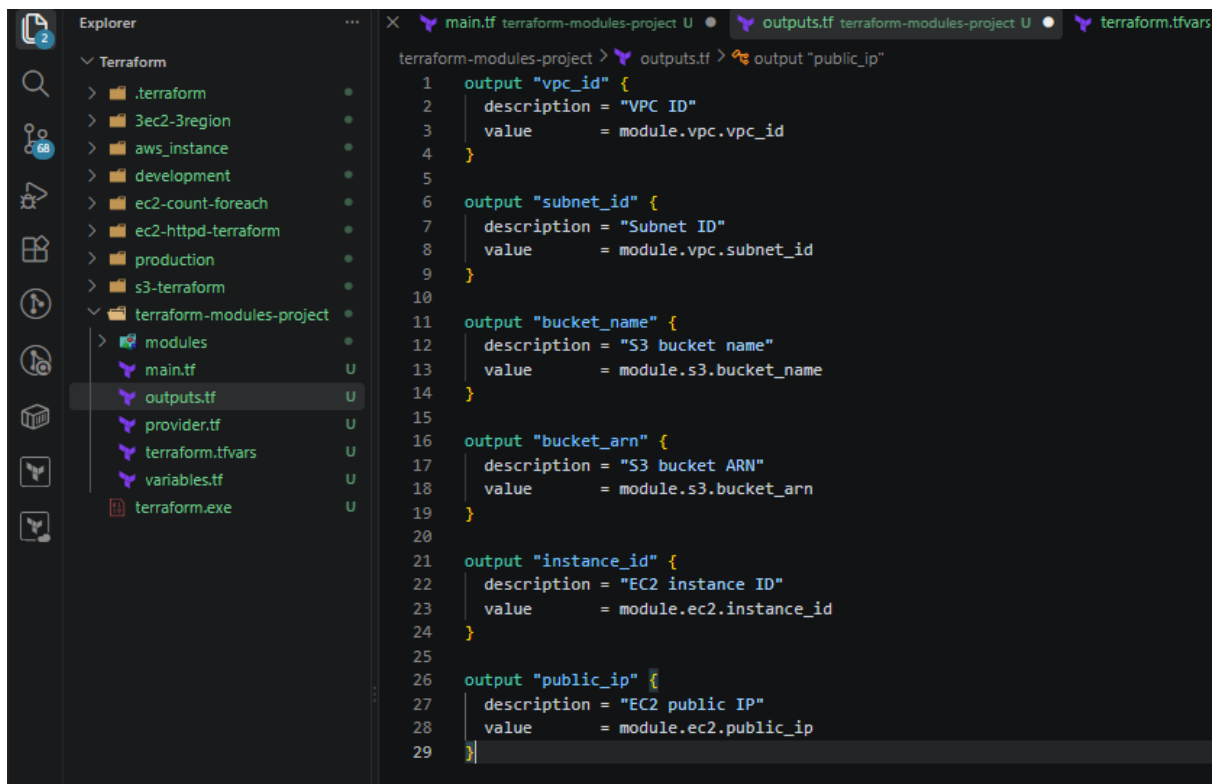
Terraform automatically understands this dependency.



14. Root outputs.tf

Create:

outputs.tf



15. Check your complete structure

Run:

ls -R

You should have:

terraform-modules-project/

|

├── provider.tf

├── variables.tf

├── main.tf

├── outputs.tf

├── terraform.tfvars

|

└── modules/

|

├── vpc/

| ├── main.tf

| ├── variables.tf

| └── outputs.tf

|

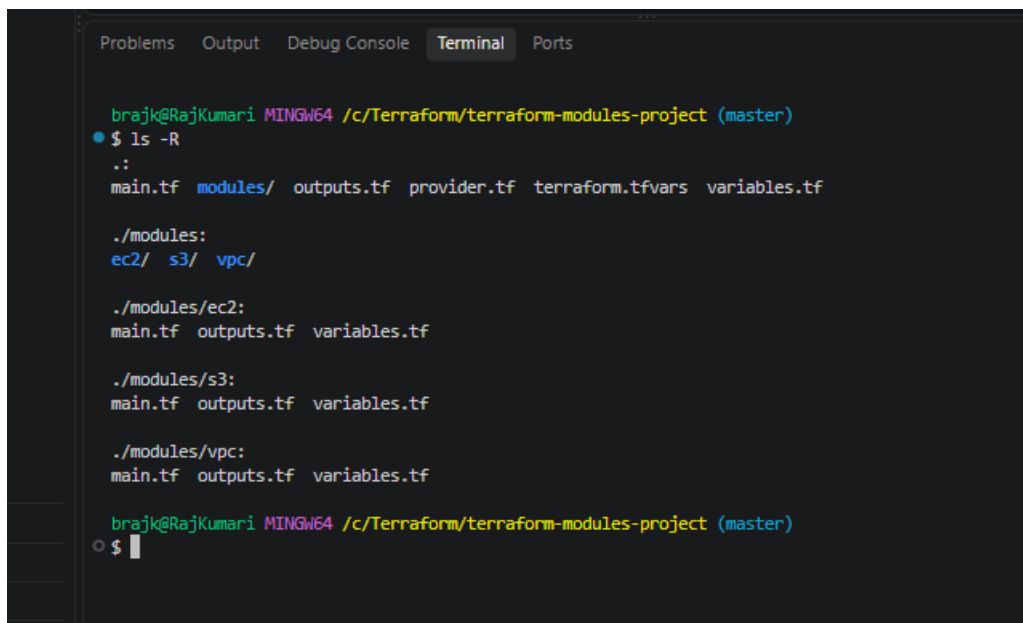
├── s3/

| ├── main.tf

| ├── variables.tf

| └── outputs.tf

```
|
|
└── ec2/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```



The screenshot shows a terminal window with the following content:

```
brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$ ls -R
.:
main.tf  modules/  outputs.tf  provider.tf  terraform.tfvars  variables.tf

./modules:
ec2/  s3/  vpc/

./modules/ec2:
main.tf  outputs.tf  variables.tf

./modules/s3:
main.tf  outputs.tf  variables.tf

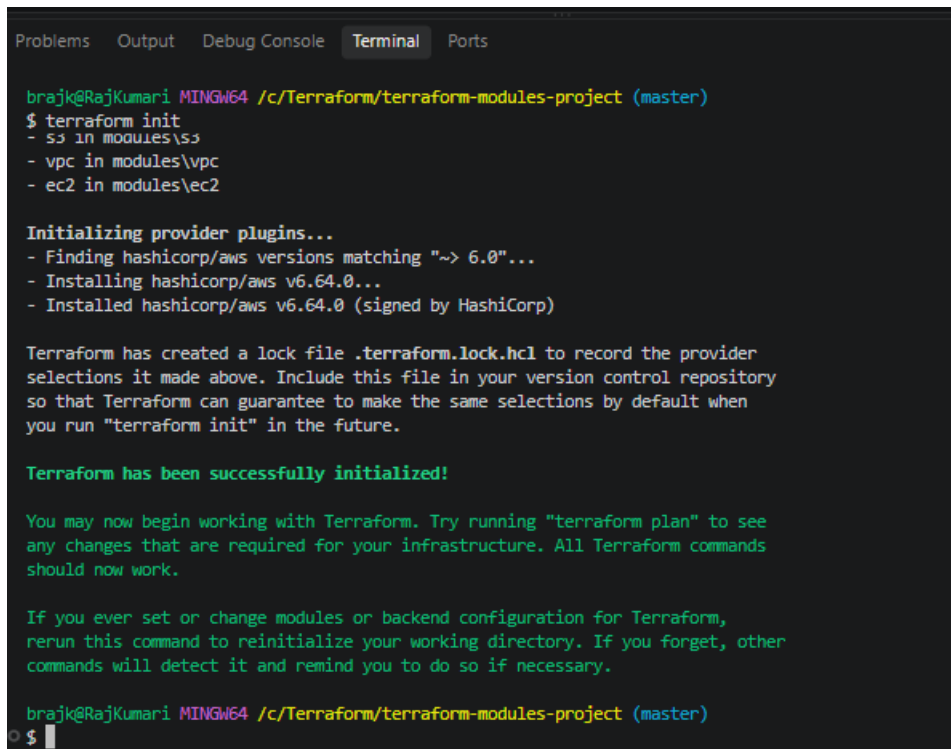
./modules/vpc:
main.tf  outputs.tf  variables.tf

brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$
```

16. Initialize Terraform

From the project root:

terraform init



```
Problems Output Debug Console Terminal Ports

brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$ terraform init
- ss in modules\ss
- vpc in modules\vpc
- ec2 in modules\ec2

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.64.0...
- Installed hashicorp/aws v6.64.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$
```

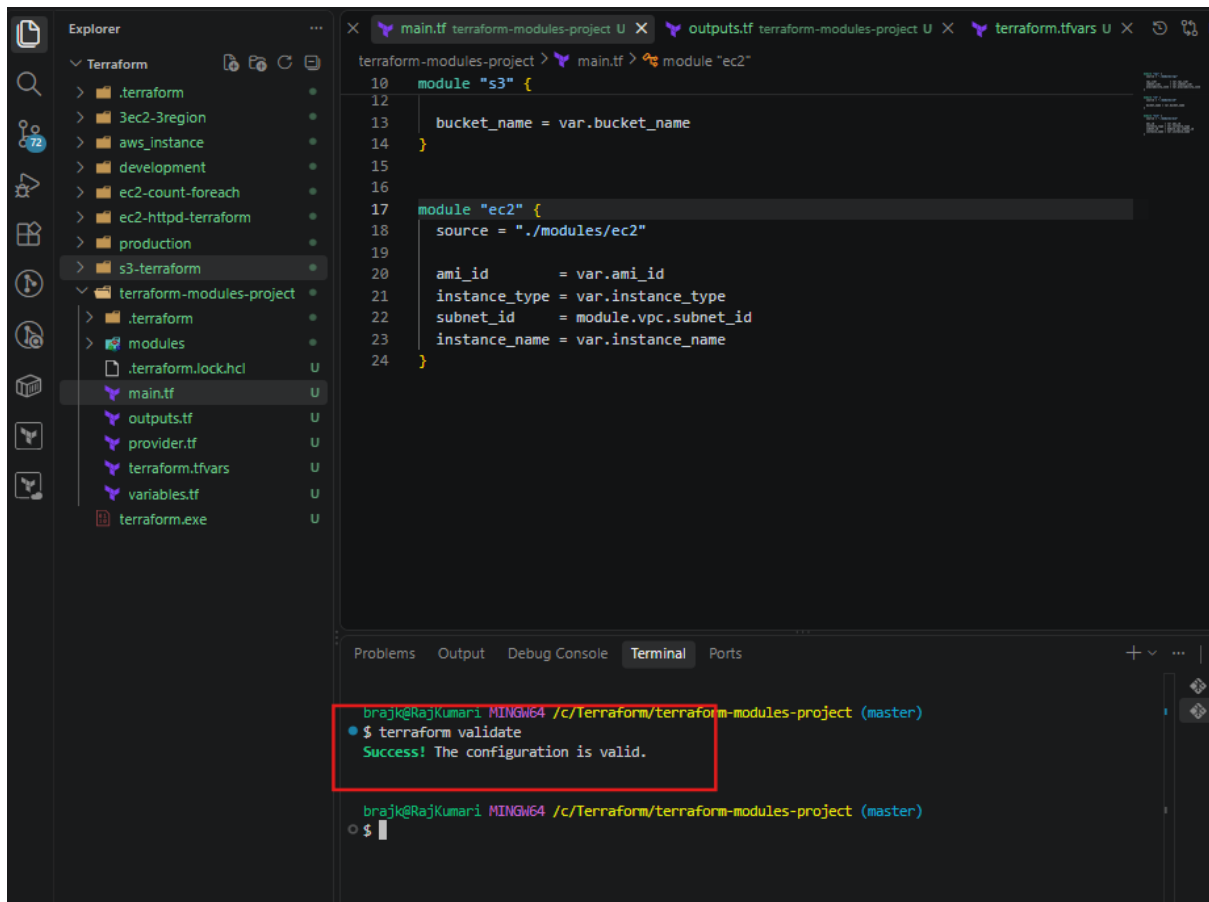
17. Validate

Run:

terraform validate

Expected:

Success! The configuration is valid.



18. Format the code

Run:

```
terraform fmt -recursive
```

This formats all Terraform files, including the modules.

19. Create execution plan

Run:

```
terraform plan
```

Terraform should show resources for:

VPC

Subnet

Security Group

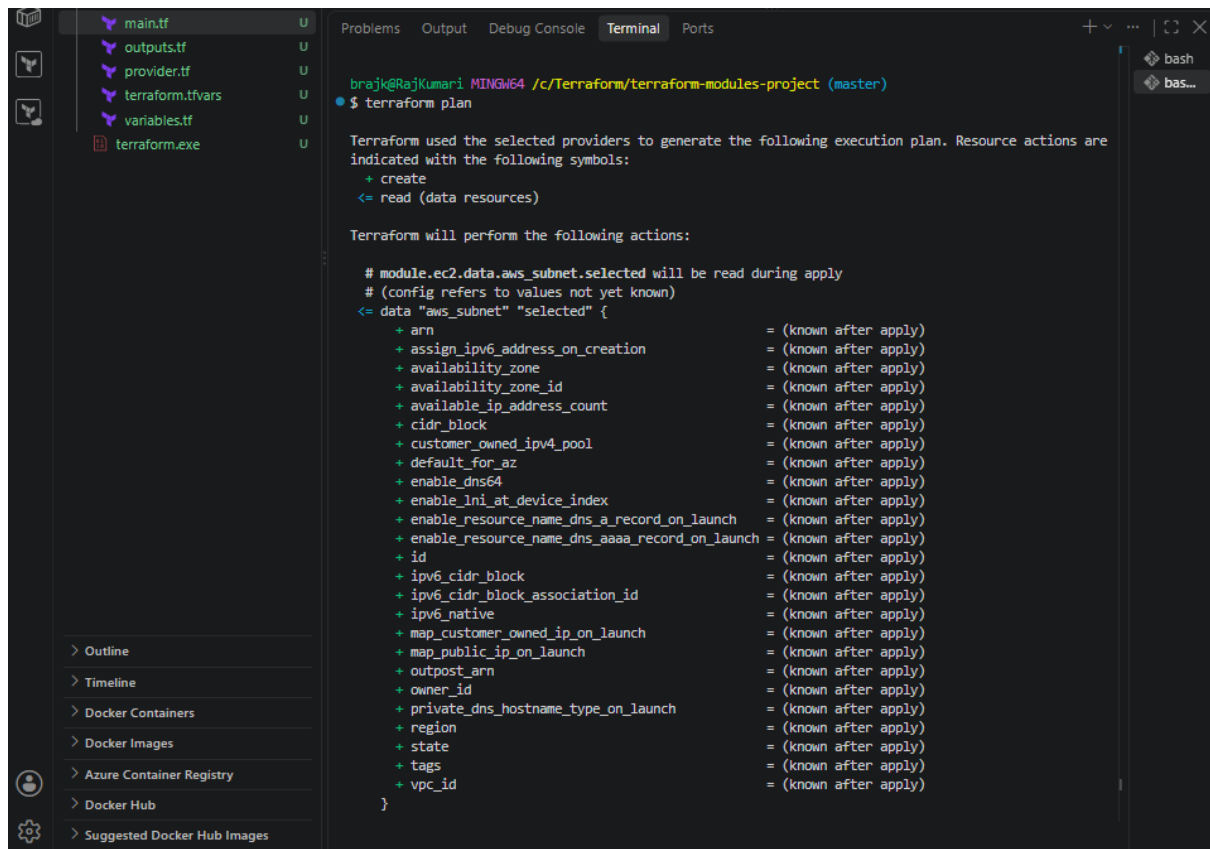
EC2

S3 Bucket

Approximately:

Plan: 5 to add, 0 to change, 0 to destroy.

The exact number can vary if you add additional resources.



The screenshot shows a VS Code editor with a file explorer on the left containing files like main.tf, outputs.tf, provider.tf, terraform.tfvars, variables.tf, and terraform.exe. The main editor area displays the output of a Terraform plan command in a terminal window. The output indicates that Terraform will perform 5 actions: 5 to add, 0 to change, and 0 to destroy. The plan details the creation of an AWS subnet resource, listing various attributes that will be set or updated.

```
brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
<= read (data resources)

Terraform will perform the following actions:

# module.ec2.data.aws_subnet.selected will be read during apply
# (config refers to values not yet known)
<= data "aws_subnet" "selected" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = (known after apply)
  + availability_zone                  = (known after apply)
  + availability_zone_id                = (known after apply)
  + available_ip_address_count          = (known after apply)
  + cidr_block                         = (known after apply)
  + customer_owned_ipv4_pool            = (known after apply)
  + default_for_az                     = (known after apply)
  + enable_dns64                       = (known after apply)
  + enable_lni_at_device_index          = (known after apply)
  + enable_resource_name_dns_a_record_on_launch = (known after apply)
  + enable_resource_name_dns_aaaa_record_on_launch = (known after apply)
  + id                                 = (known after apply)
  + ipv6_cidr_block                    = (known after apply)
  + ipv6_cidr_block_association_id      = (known after apply)
  + ipv6_native                        = (known after apply)
  + map_customer_owned_ip_on_launch     = (known after apply)
  + map_public_ip_on_launch            = (known after apply)
  + outpost_arn                        = (known after apply)
  + owner_id                           = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + region                             = (known after apply)
  + state                              = (known after apply)
  + tags                               = (known after apply)
  + vpc_id                             = (known after apply)
}
```

```
U Problems Output Debug Console Terminal Ports
U
U brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
U $ terraform plan
U   + vpc_id = (known after apply)
U   }
U
U   # module.ec2.data.aws_vpc.selected will be read during apply
U   # (config refers to values not yet known)
U   <= data "aws_vpc" "selected" {
U     + arn = (known after apply)
U     + cidr_block = (known after apply)
U     + cidr_block_associations = (known after apply)
U     + default = (known after apply)
U     + dhcp_options_id = (known after apply)
U     + enable_dns_hostnames = (known after apply)
U     + enable_dns_support = (known after apply)
U     + enable_network_address_usage_metrics = (known after apply)
U     + id = (known after apply)
U     + instance_tenancy = (known after apply)
U     + ipv6_association_id = (known after apply)
U     + ipv6_cidr_block = (known after apply)
U     + ipv6_cidr_block_associations = (known after apply)
U     + main_route_table_id = (known after apply)
U     + owner_id = (known after apply)
U     + region = (known after apply)
U     + state = (known after apply)
U     + tags = (known after apply)
U   }
U
U   # module.ec2.aws_instance.this will be created
U   + resource "aws_instance" "this" {
U     + ami = "ami-0354c98ae10b02961"
U     + arn = (known after apply)
U     + associate_public_ip_address = (known after apply)
U     + availability_zone = (known after apply)
U     + disable_api_stop = (known after apply)
U     + disable_api_termination = (known after apply)
U     + ebs_optimized = (known after apply)
U     + enable_primary_ipv6 = (known after apply)
U     + force_destroy = false
U     + get_password_data = false
U     + host_id = (known after apply)
```

The screenshot shows a VS Code editor with a terminal window open. The terminal displays the output of the command `terraform plan`. The output indicates that a new AWS VPC resource will be created. The plan shows 5 resources to be added, 0 to be changed, and 0 to be destroyed. The changes to outputs section lists several values that will be known after apply, including bucket_arn, bucket_name, instance_id, public_ip, subnet_id, and vpc_id.

```
brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$ terraform plan

# module.vpc.aws_vpc.this will be created
+ resource "aws_vpc" "this" {
  + arn                               = (known after apply)
  + cidr_block                        = "10.0.0.0/16"
  + default_network_acl_id           = (known after apply)
  + default_route_table_id           = (known after apply)
  + default_security_group_id         = (known after apply)
  + dhcp_options_id                  = (known after apply)
  + enable_dns_hostnames              = (known after apply)
  + enable_dns_support               = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                               = (known after apply)
  + instance_tenancy                 = "default"
  + ipv6_association_id              = (known after apply)
  + ipv6_cidr_block                  = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id              = (known after apply)
  + owner_id                         = (known after apply)
  + region                           = "us-east-1"
  + tags                             = {
    + "Name" = "Terraform-VPC"
  }
  + tags_all                         = {
    + "Name" = "Terraform-VPC"
  }
}

Plan: 5 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ bucket_arn = (known after apply)
+ bucket_name = "terraform-modules-raji-2026-12345"
+ instance_id = (known after apply)
+ public_ip   = (known after apply)
+ subnet_id   = (known after apply)
+ vpc_id      = (known after apply)
```

20. Apply the configuration

Run:

terraform apply

Terraform asks:

Do you want to perform these actions?

Enter:

Yes

Terraform will create:

AWS

|

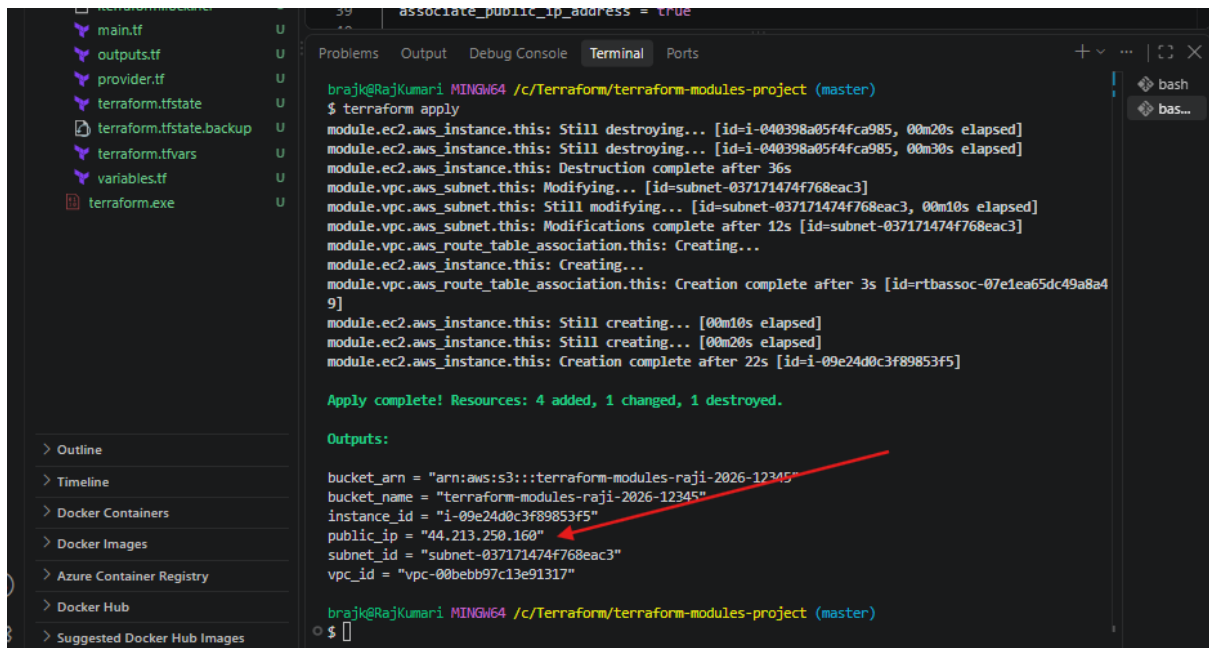
|— VPC

| |— Subnet

| |— Security Group

| — EC2

| — S3 Bucket



```
brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$ terraform apply
module.ec2.aws_instance.this: Still destroying... [id=i-040398a05f4fca985, 00m20s elapsed]
module.ec2.aws_instance.this: Still destroying... [id=i-040398a05f4fca985, 00m30s elapsed]
module.ec2.aws_instance.this: Destruction complete after 36s
module.vpc.aws_subnet.this: Modifying... [id=subnet-037171474f768eac3]
module.vpc.aws_subnet.this: Still modifying... [id=subnet-037171474f768eac3, 00m10s elapsed]
module.vpc.aws_subnet.this: Modifications complete after 12s [id=subnet-037171474f768eac3]
module.vpc.aws_route_table_association.this: Creating...
module.ec2.aws_instance.this: Creating...
module.vpc.aws_route_table_association.this: Creation complete after 3s [id=rtbassoc-07e1ea65dc49a8a49]
module.ec2.aws_instance.this: Still creating... [00m10s elapsed]
module.ec2.aws_instance.this: Still creating... [00m20s elapsed]
module.ec2.aws_instance.this: Creation complete after 22s [id=i-09e24d0c3f89853f5]

Apply complete! Resources: 4 added, 1 changed, 1 destroyed.

Outputs:

bucket_arn = "arn:aws:s3:::terraform-modules-raji-2026-12345"
bucket_name = "terraform-modules-raji-2026-12345"
instance_id = "i-09e24d0c3f89853f5"
public_ip = "44.213.250.160"
subnet_id = "subnet-037171474f768eac3"
vpc_id = "vpc-00bebb97c13e91317"

brajk@RajKumari MINGW64 /c/Terraform/terraform-modules-project (master)
$
```

22. Verify in AWS Console

VPC

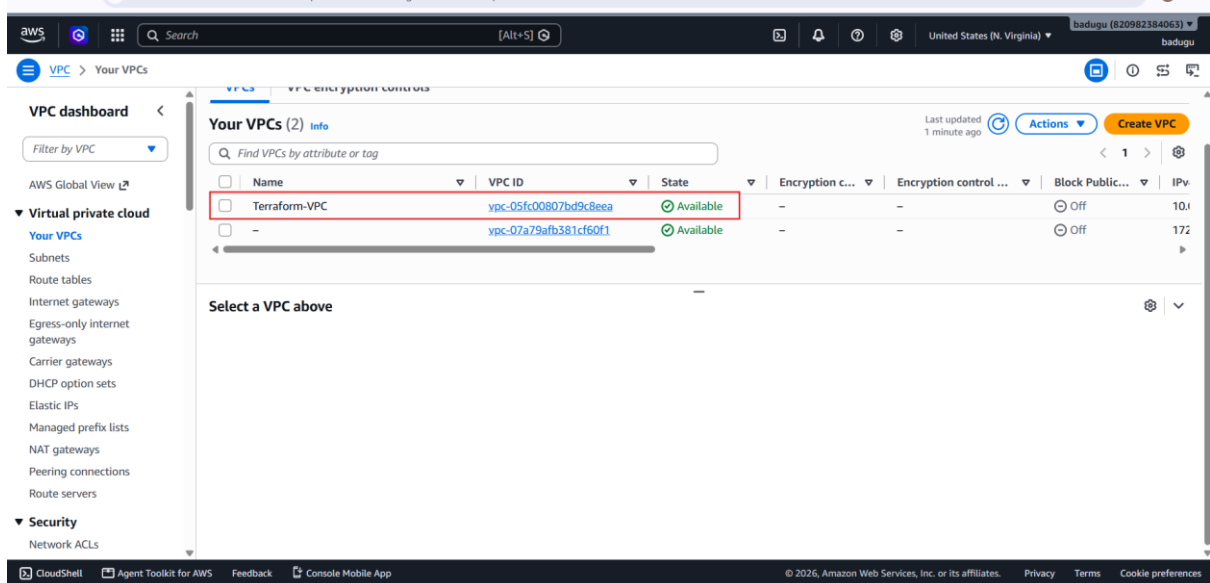
Go to:

AWS Console → VPC → Your VPCs

You should find:

Terraform-VPC

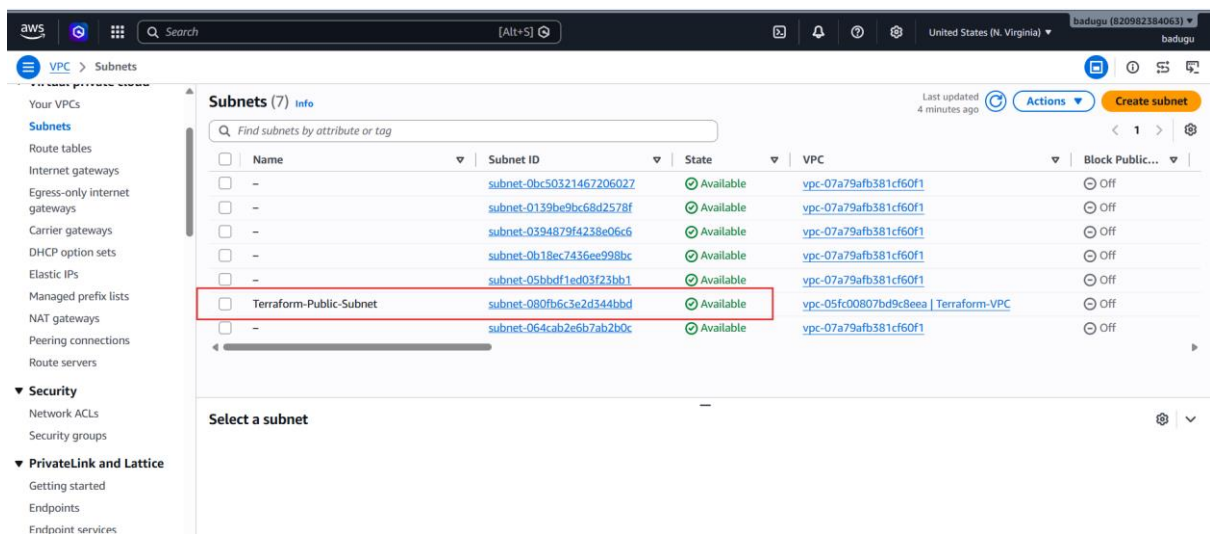
Then check:



VPC → Subnets

You should find:

Terraform-Subnet



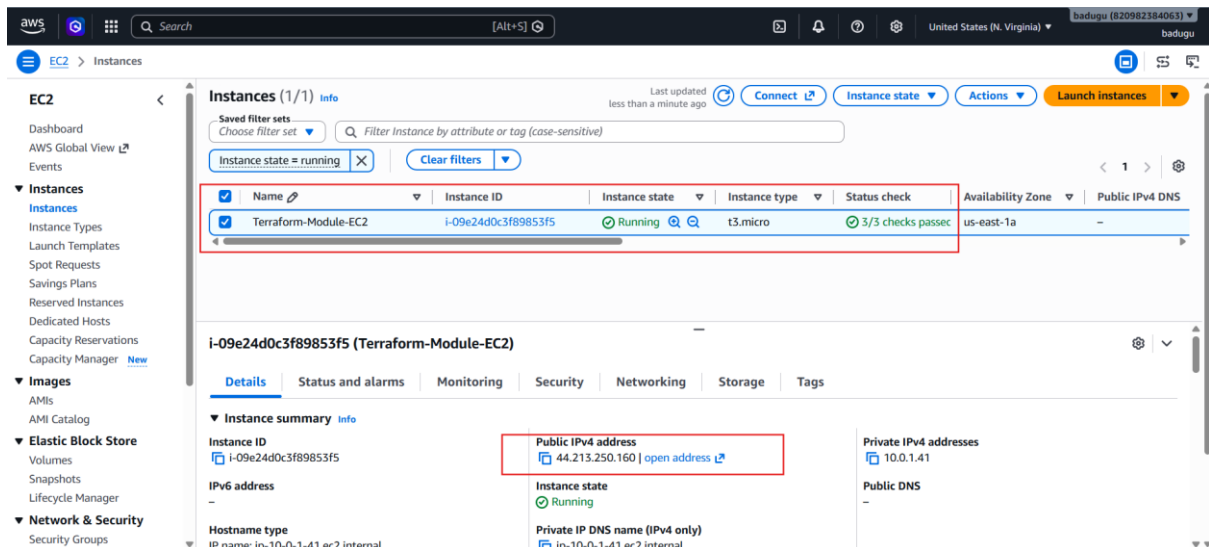
EC2

Go to:

AWS Console → EC2 → Instances

You should see:

Terraform-Module-EC2

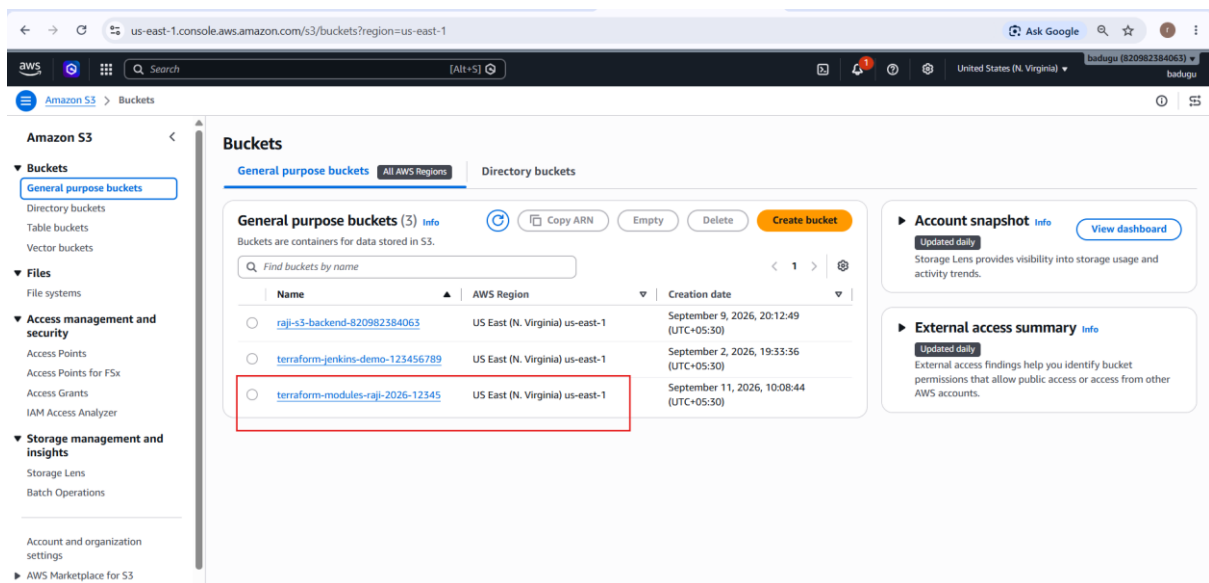


S3

Go to:

AWS Console → S3 → General purpose buckets

You should see your bucket.



4. Provision EC2 for 3 different environments (Dev, Staging, and Prod) using Terraform workspaces.

Terraform: EC2 for Dev, Staging & Prod Using Workspaces

1. Project Structure

Create a directory:

terraform-workspace-ec2/

|

├── provider.tf

├── variables.tf

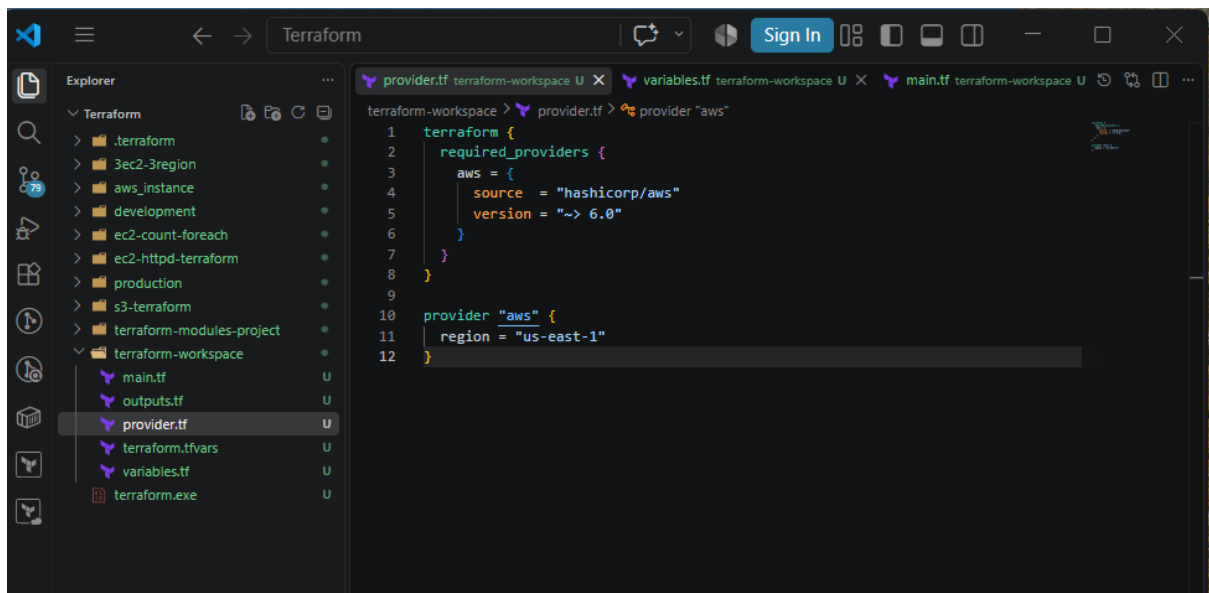
├── main.tf

├── outputs.tf

└── terraform.tfvars

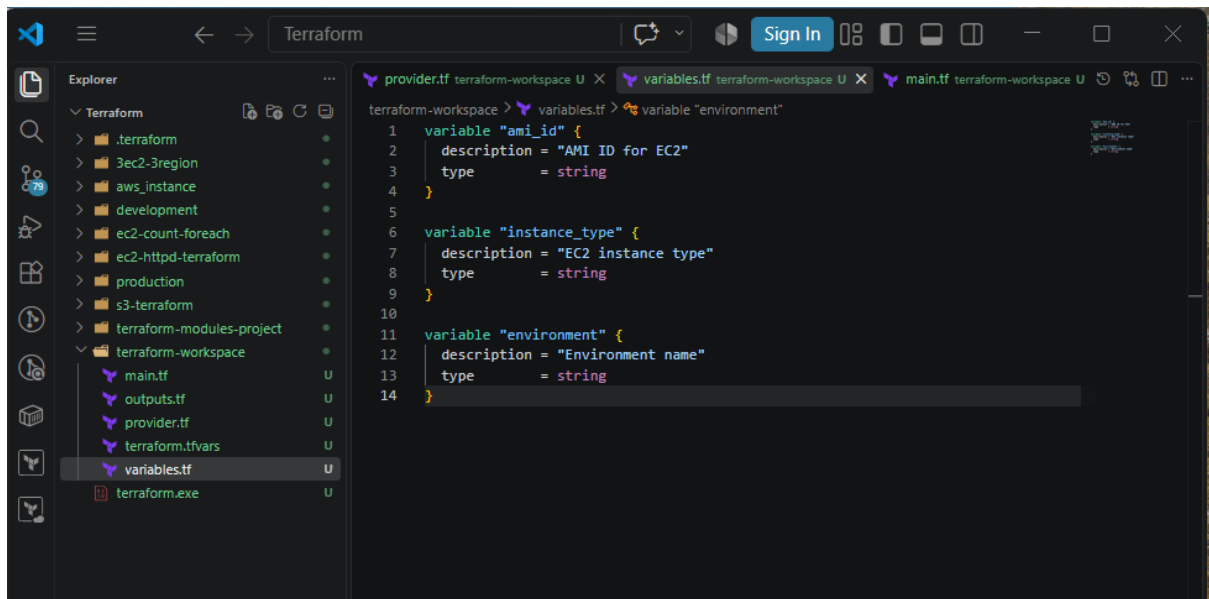
2. provider.tf

Configure AWS provider.



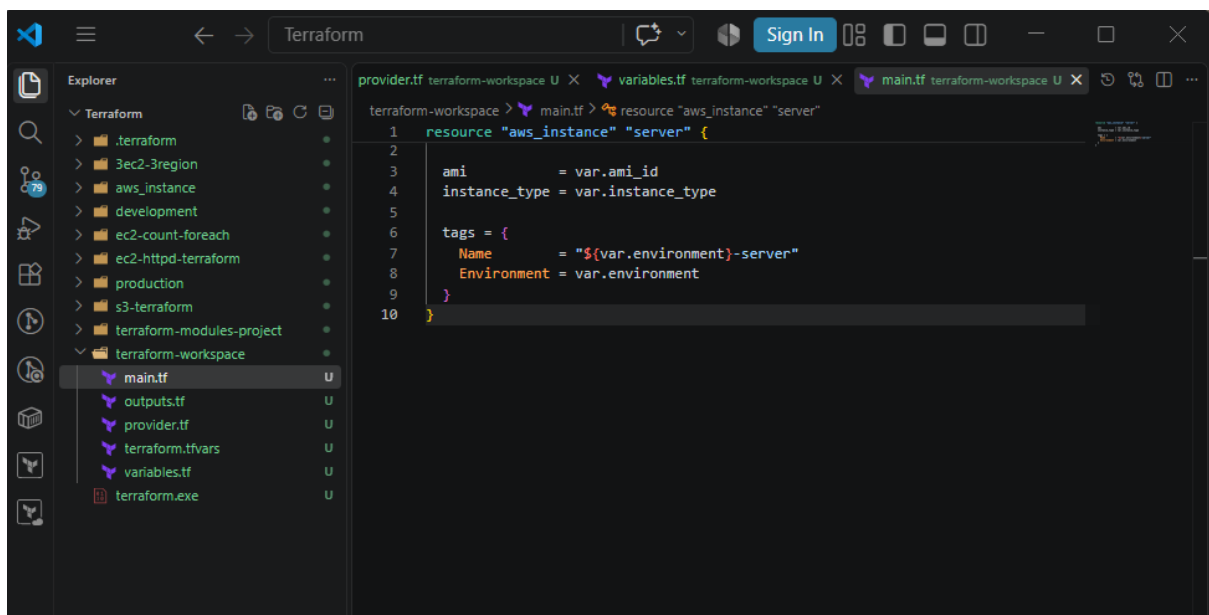
3. variables.tf

Define the variables required for EC2.



4. main.tf

Create the EC2 instance.

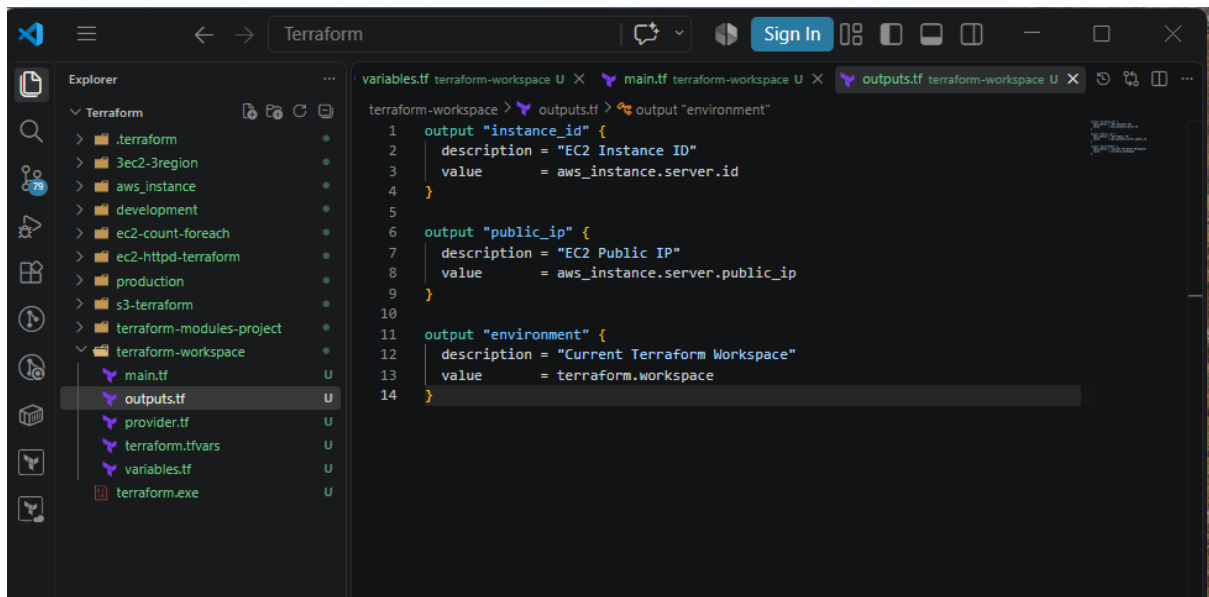


For example:

- Dev → dev-server
- Staging → staging-server
- Prod → prod-server

5. outputs.tf

Display the EC2 information.



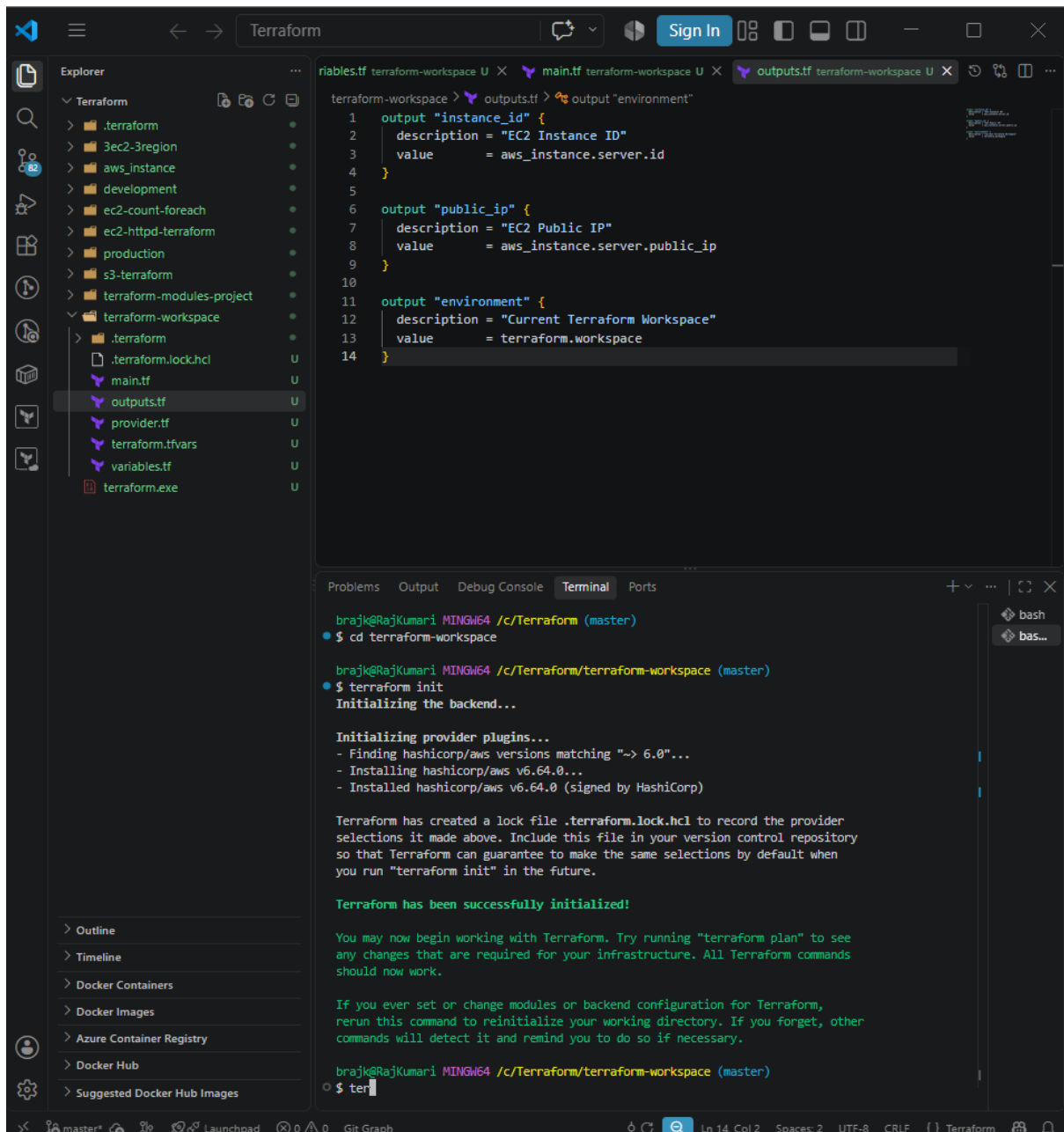
6. Initialize Terraform

From the project directory:

```
terraform init
```

Expected:

Terraform has been successfully initialized!



7. Create the Workspaces

First check existing workspaces:

terraform workspace list

Initially you may see:

* default

Create the three environments:

terraform workspace new dev

terraform workspace new staging

```
terraform workspace new prod
```

Now:

```
terraform workspace list
```

You should see:

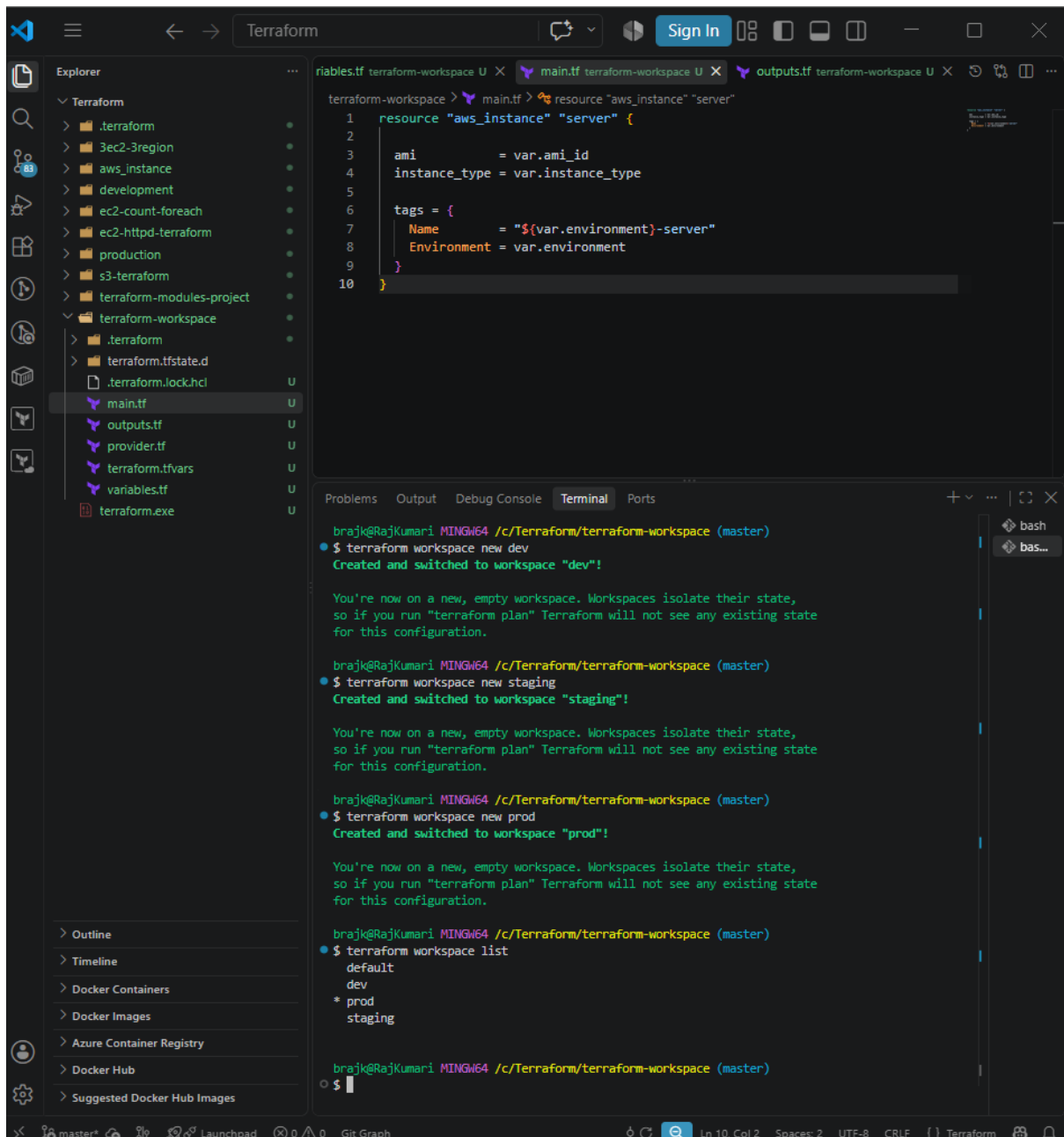
```
default
```

```
dev
```

```
staging
```

```
* prod
```

The * means the current workspace.



8. Switch to Dev Workspace

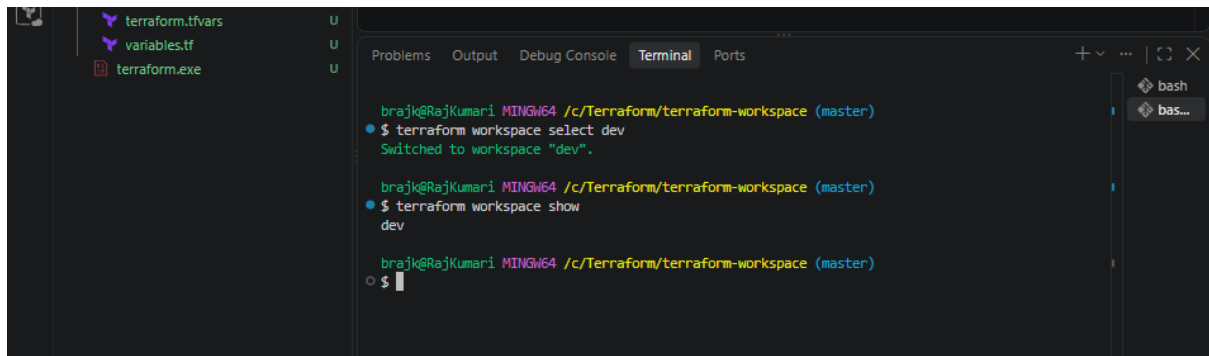
terraform workspace select dev

Check:

terraform workspace show

Output:

dev



The screenshot shows a VS Code editor with a file explorer on the left containing 'terraform.tfvars', 'variables.tf', and 'terraform.exe'. The main area is a terminal window with the following commands and output:

```
brajk@RajKumari MINGW64 /c/Terraform/terraform-workspace (master)
$ terraform workspace select dev
Switched to workspace "dev".

brajk@RajKumari MINGW64 /c/Terraform/terraform-workspace (master)
$ terraform workspace show
dev

brajk@RajKumari MINGW64 /c/Terraform/terraform-workspace (master)
$
```

9. Provision Dev EC2

You can pass the values during terraform apply.

For example:

terraform apply \

-var="ami_id=ami-xxxxxxxxxxxxxxxxxxxx" \

-var="instance_type=t2.micro" \

-var="environment=dev"

Terraform asks:

Do you want to perform these actions?

Enter a value:

Enter:

yes

Dev EC2 will be created.

10. Check Dev Output

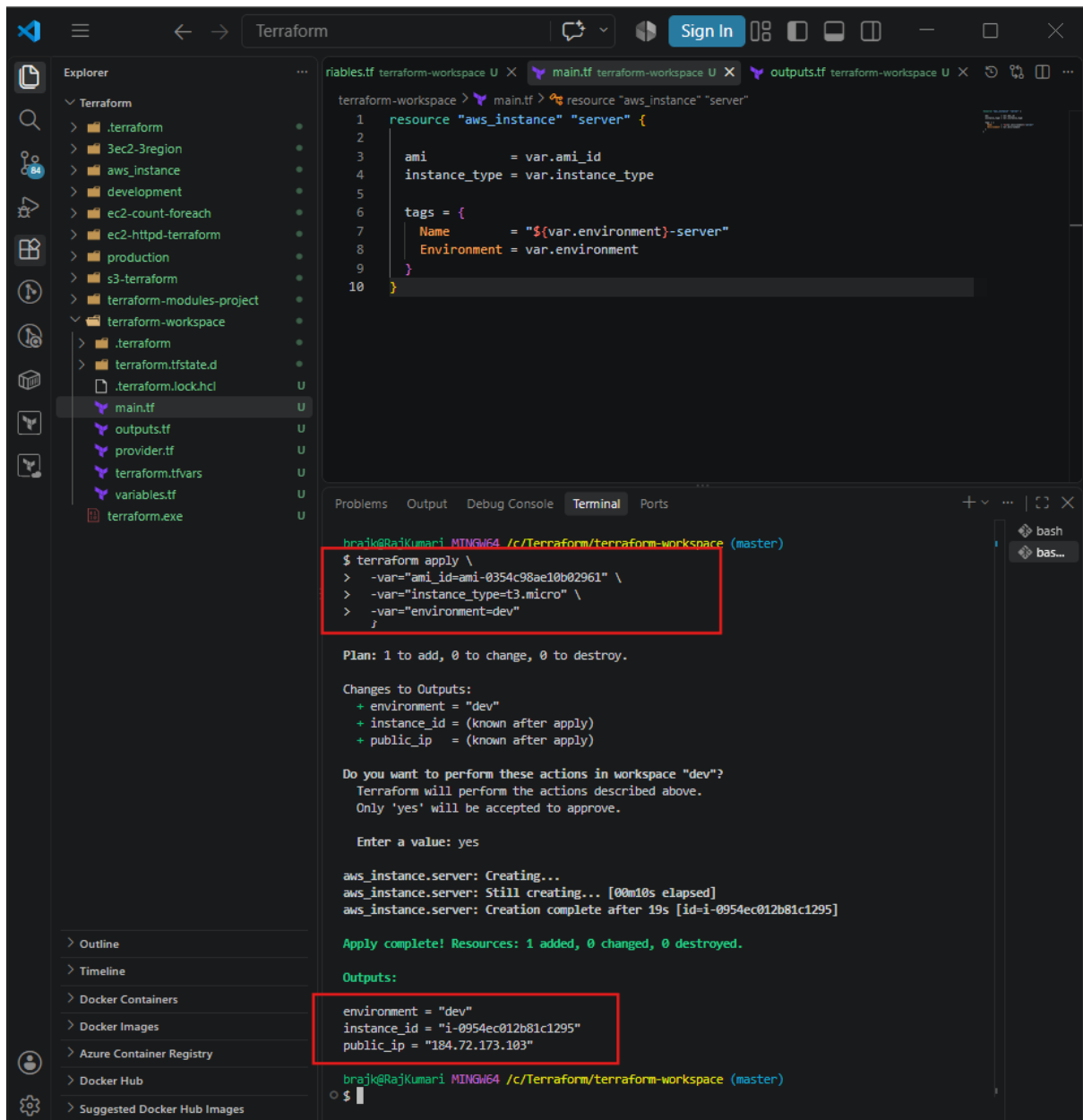
terraform output

Example:

environment = "dev"

instance_id = "i-0123456789abcdef0"

public_ip = "54.xx.xx.xx"



11. Switch to Staging

Now switch workspace:

terraform workspace select staging

Verify:

terraform workspace show

Output:

staging

Create the staging EC2:

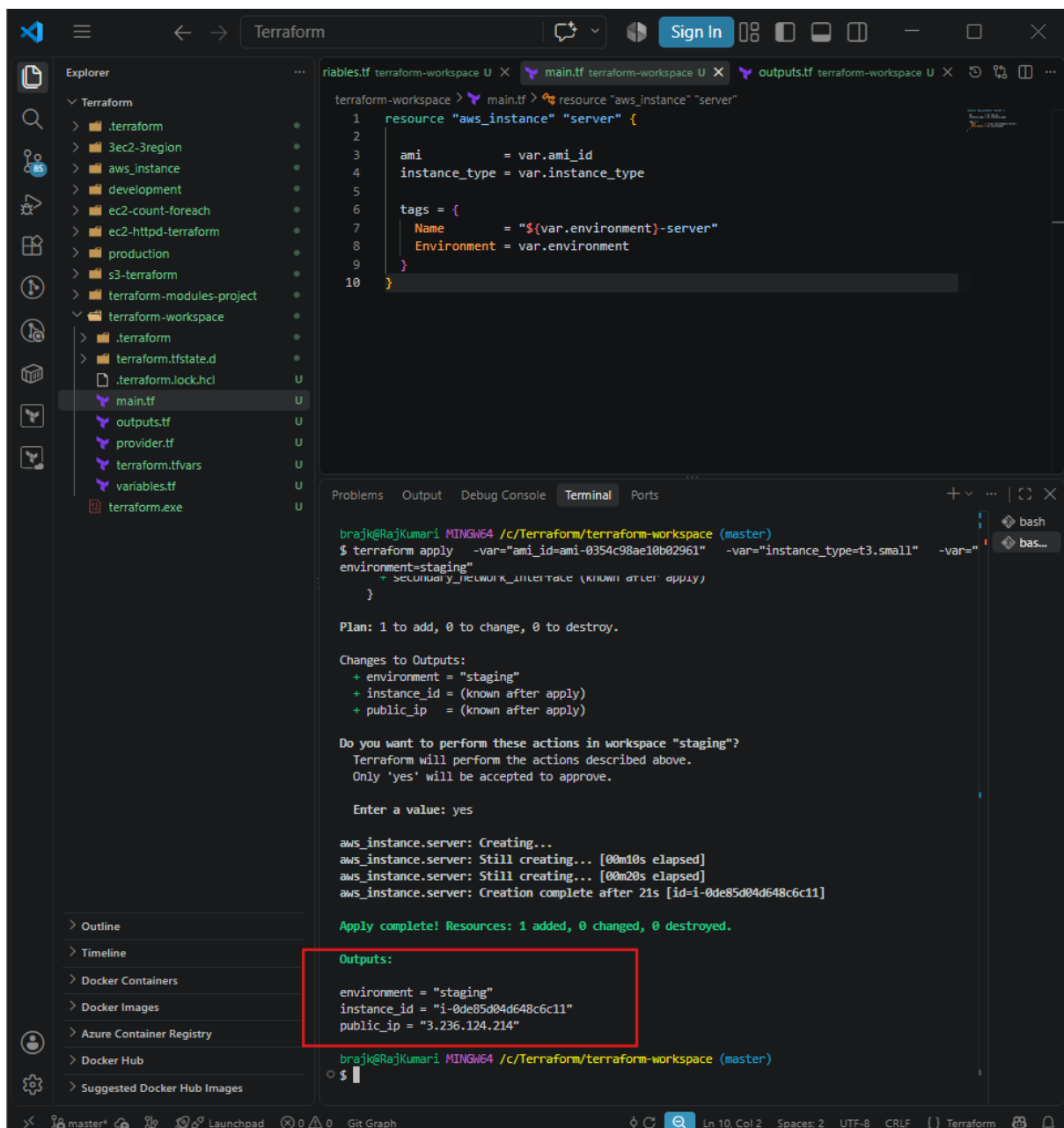
terraform apply \

-var="ami_id=ami-xxxxxxxxxxxxxxxxxxxxxx" \

-var="instance_type=t2.small" \

-var="environment=staging"

Now Terraform creates a **separate EC2 instance** for staging.



The screenshot shows a VS Code editor with a Terraform workspace. The Explorer pane on the left shows the file structure, including `main.tf`, `outputs.tf`, `provider.tf`, `terraform.tfvars`, `variables.tf`, and `terraform.exe`. The main editor displays the `main.tf` file with the following content:

```
resource "aws_instance" "server" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name        = "${var.environment}-server"
    Environment = var.environment
  }
}
```

The terminal pane at the bottom shows the execution of the `terraform apply` command with the specified variables. The output indicates that the instance was created successfully. A red box highlights the `Outputs:` section, which shows the following values:

```
environment = "staging"
instance_id = "i-0de85d04d648c6c11"
public_ip   = "3.236.124.214"
```

14. Better Approach: Automatically Change Configuration by Workspace

Instead of manually providing:

```
-var="environment=dev"
```

we can use:

```
terraform.workspace
```

This automatically gives us the current workspace.

For example, change main.tf to:

```
locals {  
    environment = terraform.workspace  
  
    instance_types = {  
        dev    = "t2.micro"  
        staging = "t2.small"  
        prod   = "t2.medium"  
    }  
}  
  
resource "aws_instance" "server" {  
  
    ami          = var.ami_id  
    instance_type = local.instance_types[terraform.workspace]  
  
    tags = {  
        Name      = "${local.environment}-server"  
        Environment = local.environment  
    }  
}
```

Now you don't need the environment variable.

15. Updated variables.tf

Use:

```
variable "ami_id" {  
    description = "AMI ID for EC2"  
    type        = string  
}
```

16. Updated outputs.tf

```
output "instance_id" {  
    value = aws_instance.server.id  
}
```

```
output "public_ip" {  
    value = aws_instance.server.public_ip  
}
```

```
output "environment" {  
    value = terraform.workspace  
}
```

```
output "instance_type" {  
    value = aws_instance.server.instance_type  
}
```

17. Now Deploy Dev

Select Dev:

```
terraform workspace select dev
```

Apply:

```
terraform apply -var="ami_id=ami-xxxxxxxxxxxxxxxxxx"
```

Terraform automatically uses:

```
terraform.workspace = dev
```

Therefore:

```
Instance Type = t2.micro
```

```
Environment = dev
```

```
Name = dev-server
```

18. Deploy Staging

```
terraform workspace select staging
```

Then:

```
terraform apply -var="ami_id=ami-xxxxxxxxxxxxxxxxxx"
```

Terraform automatically uses:

```
terraform.workspace = staging
```

Therefore:

```
Instance Type = t2.small
```

```
Environment = staging
```

```
Name = staging-server
```

19. Deploy Production

```
terraform workspace select prod
```

Then:

```
terraform apply -var="ami_id=ami-xxxxxxxxxxxxxxxxxx"
```

Terraform automatically uses:

terraform.workspace = prod

Therefore:

Instance Type = t2.medium

Environment = prod

Name = prod-server

20. Final AWS Result

In the EC2 console you should have something like:

Workspace EC2 Name		Instance Type
dev	dev-server	t2.micro
staging	staging-server	t2.small
prod	prod-server	t2.medium

The important thing is that **the same Terraform code** creates all three environments.

21. How Terraform Knows Which Environment to Use

This is the key interview concept.

When you run:

```
terraform workspace select dev
```

Terraform sets:

```
terraform.workspace
```

to:

```
dev
```

When you select:

```
terraform workspace select staging
```

it becomes:

staging

And:

terraform workspace select prod

becomes:

prod

So this code:

```
instance_type = local.instance_types[terraform.workspace]
```

works like:

Workspace	Instance Type
dev	t2.micro
staging	t2.small
prod	t2.medium

22. How to Destroy Only Dev

Suppose you want to delete **only Dev**.

First:

```
terraform workspace select dev
```

Then:

```
terraform destroy -var="ami_id=ami-xxxxxxxxxxxxxxxxxx"
```

This destroys the resources managed by the **Dev workspace state**.

Your staging and production resources are not destroyed.

23. Important Interview Question

Q: Why use Terraform workspaces?

Answer:

Terraform workspaces allow us to use the same Terraform configuration for multiple environments while maintaining separate state files for each workspace. For example, we can create dev, staging, and prod workspaces and provision different infrastructure configurations for each environment.

Q: What is terraform.workspace?**Answer:**

terraform.workspace is a built-in Terraform expression that returns the name of the currently selected workspace.

For example:

```
terraform.workspace
```

returns:

```
dev
```

when the dev workspace is selected.

24. Complete Final Code**provider.tf**

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}  
  
provider "aws" {
```

```
    region = "us-east-1"
}
```

variables.tf

```
variable "ami_id" {
    description = "AMI ID for EC2"
    type        = string
}
```

main.tf

```
locals {
    environment = terraform.workspace

    instance_types = {
        dev    = "t2.micro"
        staging = "t2.small"
        prod   = "t2.medium"
    }
}

resource "aws_instance" "server" {

    ami        = var.ami_id
    instance_type = local.instance_types[terraform.workspace]

    tags = {
        Name        = "${local.environment}-server"
        Environment = local.environment
    }
}
```

```
}  
  
}  
  
outputs.tf  
  
output "instance_id" {  
    value = aws_instance.server.id  
}  
  
output "public_ip" {  
    value = aws_instance.server.public_ip  
}  
  
output "environment" {  
    value = terraform.workspace  
}  
  
output "instance_type" {  
    value = aws_instance.server.instance_type  
}
```

Complete Execution Flow

terraform init

terraform workspace new dev

terraform workspace new staging

terraform workspace new prod

Then:

terraform workspace select dev

```
terraform apply -var="ami_id=YOUR_AMI_ID"
```

```
terraform workspace select staging
```

```
terraform apply -var="ami_id=YOUR_AMI_ID"
```

```
terraform workspace select prod
```

```
terraform apply -var="ami_id=YOUR_AMI_ID"
```

Finally:

```
terraform workspace list
```

You will have:

default

dev

staging

* prod

One code → three workspaces → three separate Terraform states → three EC2 environments.

